

UNIVERSIDADE FEDERAL RURAL DO SEMI-ÁRIDO
PRÓ-REITORIA DE GRADUAÇÃO
DEPARTAMENTO DE ENGENHARIAS E TECNOLOGIA
CURSO INTERDISCIPLINAR EM TECNOLOGIA DA INFORMAÇÃO

MARIA LUMA DA SILVA OLIVEIRA

Um estudo sobre a aplicação de LLMs no teste de software

PAU DOS FERROS - RN
2025

Maria Luma da Silva Oliveira

Um estudo sobre a aplicação de LLMs no teste de software

Monografia apresentada à Universidade Federal Rural do Semi-Árido como requisito para obtenção do título de Bacharel em Tecnologia da Informação.

Orientador: Prof. Dr. João Batista de Souza Neto.

PAU DOS FERROS - RN

2025

© Todos os direitos estão reservados a Universidade Federal Rural do Semi-Árido. O conteúdo desta obra é de inteira responsabilidade do (a) autor (a), sendo o mesmo, passível de sanções administrativas ou penais, caso sejam infringidas as leis que regulamentam a Propriedade Intelectual, respectivamente, Patentes: Lei n° 9.279/1996 e Direitos Autorais: Lei n° 9.610/1998. O conteúdo desta obra tomar-se-á de domínio público após a data de defesa e homologação da sua respectiva ata. A mesma poderá servir de base literária para novas pesquisas, desde que a obra e seu (a) respectivo (a) autor (a) sejam devidamente citados e mencionados os seus créditos bibliográficos.

O48e Oliveira, Maria Luma da Silva.
Um estudo sobre a aplicação de LLMs no teste de software / Maria Luma da Silva Oliveira. - 2025.
58 f. : il.

Orientador: João Batista de Souza Neto.
Monografia (graduação) - Universidade Federal Rural do Semi-árido, Curso de Tecnologia da Informação, 2025.

1. LLM. 2. Teste de software. 3. Engenharia de prompt. 4. JaCoCo. 5. PIT. I. Souza Neto, João Batista de , orient. II. Título.

Ficha catalográfica elaborada por sistema gerador automático em conformidade com AACR2 e os dados fornecidos pelo autor(a).
Biblioteca Campus Pau dos Ferros
Bibliotecária: Erlanda Maria Lopes da Silva CRB: 15/966

O serviço de Geração Automática de Ficha Catalográfica para Trabalhos de Conclusão de Curso (TCC´s) foi desenvolvido pelo Instituto de Ciências Matemáticas e de Computação da Universidade de São Paulo (USP) e gentilmente cedido para o Sistema de Bibliotecas da Universidade Federal Rural do Semi-Árido (SISBI-UFERSA), sendo customizado pela Superintendência de Tecnologia da Informação e Comunicação (SUTIC) sob orientação dos bibliotecários da instituição para ser adaptado às necessidades dos alunos dos Cursos de Graduação e Programas de Pós-Graduação da Universidade.

Maria Luma da Silva Oliveira

Um estudo sobre a aplicação de LLMs no teste software

Monografia apresentada à Universidade Federal Rural do Semi-Árido como requisito para obtenção do título de Bacharel em Tecnologia da Informação.

Defendida em: 04 / 08 / 2025.

BANCA EXAMINADORA

_____
João Batista de Souza Neto, Prof. Dr. (UFERSA)
Presidente

_____
Bruno Borges da Silva, Prof. Esp. (UFERSA)
Membro Examinador

_____
George Felipe Fernandes Vieira, Prof. Esp. (UFERSA)
Membro Examinador

_____

DEDICATÓRIA

Dedico este trabalho aos meus pais, Maria Sandra e Juraci Araújo. Vocês foram e são os pilares da minha vida, as luzes que iluminaram os caminhos mais incertos e a força que me impulsionou a cada passo. Por todo o amor incondicional, pelos sacrifícios, pela paciência e pela crença inabalável em meu potencial, minha eterna gratidão. Este TCC é fruto do apoio e dos valores que vocês me ensinaram.

AGRADECIMENTOS

A conclusão desta jornada, sinto um profundo dever de gratidão a todos que, de alguma forma, tornaram este trabalho possível. Primeiramente, agradeço a Deus, pela força, sabedoria e pela inabalável que me guiaram em cada passo e em cada desafio enfrentado.

Aos meus pais, Maria Sandra e Juraci Araujo, por serem meu alicerce, minha maior inspiração e por todo o amor, apoio incondicional e sacrificios que me permitiram chegar até aqui. Esta conquista é, em grande parte, de vocês.

À minha irmã, Lorena Oliveira, e ao meu irmão Bento Soares, pelo companheirismo, pelas risadas e por estarem sempre ao meu lado. A presença de vocês foi um porto seguro nesta trajetória.

Ao meu orientador, Prof. Dr. João Batista, pela paciência, por suas orientações sempre precisas e pela dedicação em compartilhar seu conhecimento, conduzindo-me com segurança durante todo o desenvolvimento deste trabalho.

Ao meu tio, Josenir Araújo, que, mesmo de forma indireta, ofereceu um suporte importante, fazendo toda a diferença.

Aos meus amigos Mateus Gomes, Iris Matias e Gerson Breno, por compartilharem comigo alegria e dificuldades. A amizades de vocês foi um bálsamo e um lembrete de que eu nunca estive sozinha.

Aos estimados membros da banca examinadora, Prof. Bruno Silva e Prof. George Vieira, pela disponibilidade, pelas contribuições valiosas e pelas sugestões que enriqueceram significativamente este estudo.

A Ex-coordenadora de assuntos estudantis (COAE), Priscila Siméia, por todo apoio constante e ajuda indispensável ao longo de todo esse processo.

As minhas colegas de quarto na residência, Karol e Jaelica, pela convivência, pelas conversas, o que ajudou a minha jornada ser mais leve.

Por fim, e não menos importante, a todos os docentes da Universidade Federal Rural do Semi-Árido (UFERSA), cuja expertise e dedicação contribuíram de forma imensurável para minha formação acadêmica e pessoal.

RESUMO

Este trabalho explora a aplicação de Modelos de Linguagem de Grande Escala (LLMs) no processo de Teste de Software, com foco na geração automatizada de artefatos de teste. O objetivo foi desenvolver e validar uma abordagem baseada em prompts genéricos e reutilizáveis para apoiar tarefas específicas em testes funcionais, testes estruturais e testes automatizados. O guia de prompts detalha a criação de casos de teste para técnicas como particionamento em classe de equivalência, análise de valor limite, cobertura de linha, cobertura de ramificação e cobertura de condições, além da geração de scripts de automação. Para avaliar a eficácia e a qualidade dos testes gerados pelos LLMs, foi adotada uma metodologia experimental de caráter quantitativo. Métricas de cobertura de código, como cobertura de instrução e cobertura de ramificação, foram coletadas utilizando a ferramenta JaCoCo. Adicionalmente, a qualidade da detecção de falhas dos testes foi verificada através de teste de mutação, com a extração da pontuação de mutação pela ferramenta PIT. Os resultados obtidos contribuem para a compreensão do potencial dos LLMs em otimizar o processo de garantia de qualidade de software, destacando tanto suas capacidades na criação de testes quanto às áreas onde a automação ainda pode ser aprimorada para garantir maior robustez e eficácia.

Palavras-chave: LLMs; teste de software; engenharia de prompt; JaCoCo; PIT.

ABSTRACT

This study investigates the application of Large Language Models (LLMs) in Software Testing, focusing on the automated generation of test artifacts. The core objective was to develop and validate an approach based on generic and reusable prompts to support specific tasks in functional testing, structural testing, and automated testing. The prompt guide details the creation of test cases for techniques such as equivalence class partitioning, boundary value analysis, line coverage, branch coverage, and condition coverage, in addition to generating test automation scripts. To evaluate the effectiveness and quality of the tests generated by the LLMs, an experimental quantitative methodology was adopted. Code coverage metrics, specifically instruction coverage and branch coverage, were collected using the JaCoCo tool. Furthermore, the fault detection capability of the tests was assessed through mutation testing, with the mutation score extracted using the PIT tool. The results contribute to understanding the potential of LLMs in optimizing the software quality assurance process, highlighting both their capabilities in test creation and areas where automation can still be improved to ensure greater robustness and effectiveness.

Keywords: LLMs; software testing; test automation; prompt engineering; JaCoCo; PIT.

LISTA DE QUADROS

Quadro 1 - Comparação dos trabalhos relacionados..... 31
Quadro 2 - Teste de particionamento em classe de equivalência..... 35
Quadro 3 - Teste de análise de valor limite..... 35
Quadro 4 - Teste de cobertura de linha..... 36
Quadro 5 - Teste de cobertura de ramificação..... 36
Quadro 6 - Teste de cobertura de condições..... 37
Quadro 7 - Gerar script de teste automatizado..... 38
Quadro 8 - Gerar script de teste automatizado a partir de uma especificação..... 38
Quadro 9: Métricas de avaliação dos testes gerados..... 42
Quadro 10: Resultados para casos sem critérios..... 50
Quadro 11: Médias por critério..... 51
Quadro 12: Impacto dos casos inválidos por programas..... 52
Quadro 13: Impacto das falhos por programas..... 53
Quadro 14: Consolidação por programa..... 54

LISTA DE ABREVIATURAS E SIGLAS

AID	Program Variants Generation, Input Generation, And Differential Testing.
API	Application Programming Interface
AVL	Análise de Valor Limite
CC	Cobertura de Condição
CL	Cobertura de Linha
CR	Cobertura de Ramificação
CT	Caso de Teste
CTF	Cobertura de Ramificação
CTI	Caso de Teste Inválidos
CW	CountWords
GUI	Graphical User Interface
IA	Inteligência Artificial
JaCoCo	Java Code Coverage
LLM	Large Language Models
NA	Nível de Automação
PCE	Particionamento de Classe de Equivalência
PIT	Parallel Isolated Test
PP	PlanningPoker
PUT	Program Under Test
QA	Quality Assurance
SB	SubstringsBetween
SC	Sem Critério
TT	TriangleType

SUMÁRIO

1 INTRODUÇÃO..... 13
1.1 Justificativa..... 14
1.2 Objetivo..... 14
1.2.1 Objetivo Geral..... 14
1.3 Organização do Trabalho..... 15
2 FUNDAMENTAÇÃO TEÓRICA..... 16
2.1 Teste de Software..... 16
2.1.1 Conceitos Fundamentais..... 16
2.1.2 Atividades do Processo de Teste de Software..... 19
2.1.3 Teste Estrutural..... 20
2.1.4 Teste Funcional..... 21
2.1.5 Testes Automatizados..... 22
2.2 Modelos De Linguagem De Grande Escala (LLMs)..... 22
2.2.1 Engenharia de Prompt..... 24
3.1 Make LLM a Testing Expert: Bringing Human-like Interaction to Mobile GUI..... 25
Testing via Functionality-aware Decisions..... 25
3.2 LLM-Powered Test Case Generation for Detecting Tricky Bugs..... 26
3.3 ChatGPT prompt patterns for improving code quality, refactoring, requirements elicitation, and software design..... 27
3.4 A prompt pattern catalog to enhance prompt engineering with ChatGPT..... 28
3.5 Large Language Model Assisted Software Engineering: Prospects, Challenges, and a Case Study Preprint..... 29
3.6 Discussões..... 29
5 DIRETRIZES PARA O USO DE LLMS NO TESTE DE SOFTWARE..... 34
5.1 Teste Funcional..... 34
5.2 Teste Estrutural..... 35
5.3 Teste Automatizados..... 37
6 EXPERIMENTO DE AVALIAÇÃO..... 40
6.1 Processo de Execução dos Experimentos..... 40
6.2 Resultados dos Experimentos..... 42
6.2.1 CountWords..... 43
6.2.1.1 Particionamento de Equivalência de Classes..... 43
6.2.1.2 Análise de Valor Limite..... 44
6.2.1.3 Cobertura de Linha..... 44
6.2.1.4 Cobertura de Ramificações..... 44
6.2.1.5 Cobertura de Condições..... 44
6.2.1.6 Sem Critérios..... 45
6.2.2 PlanningPoker..... 45
6.2.2.1 Particionamento de Equivalência de Classes..... 45
6.2.2.2 Análise de Valor Limite..... 45
6.2.2.3 Cobertura de Linha..... 46

6.2.2.4 Cobertura de Ramificações..... 46
6.2.2.5 Cobertura de Condições..... 46
6.2.2.6 Sem Critérios..... 46
6.2.3 SubstringsBetween..... 47
6.2.3.1 Particionamento de Equivalência de Classes..... 47
6.2.3.2 Análise de Valor Limite..... 47
6.2.3.3 Cobertura de Linha..... 47
6.2.3.4 Cobertura de Ramificações..... 48
6.2.3.5 Cobertura de Condições..... 48
6.2.3.6 Sem Critérios..... 48
6.2.4 TriangleType..... 48
6.2.4.1 Particionamento de Equivalência de Classes..... 48
6.2.4.2 Análise de Valor Limite..... 49
6.2.4.3 Cobertura de Linha..... 49
6.2.4.4 Cobertura de Ramificações..... 49
6.2.4.5 Cobertura de Condições..... 49
6.2.4.6 Sem Critérios..... 49
6.3 Análise dos Resultados..... 50
6.4 Limitações do Estudo..... 57
7 CONSIDERAÇÕES FINAIS..... 58
7.1 Trabalhos Futuros..... 58
8 REFERÊNCIAS..... 60

1 INTRODUÇÃO

A Engenharia de Software é a área responsável por aplicar princípios, métodos e ferramentas para o desenvolvimento sistemático, eficiente e de alta qualidade de sistemas de software, indo muito além da simples programação. Sua importância é evidente no mundo moderno, onde praticamente todos os setores - como transporte, saúde, comunicação, finanças e entretenimento - dependem de software para funcionar. Conforme destaca Sommerville (2011), sistemas de software controlam infraestrutura críticas, dispositivos eletrônicos e serviços essenciais, sendo fundamentais para a operação de sociedades nacionais e globais. A engenharia de software permite lidar com a complexidade crescente desses sistemas, oferecendo manutenção e escalabilidade aos produtos.

O Teste de Software é umas das etapas essenciais no desenvolvimento do sistemas, pois garante que o produto atenda aos requisitos definidos e funcione corretamente sob diferentes condições. Como destaca Sommerville (2011), observa que o teste tem o objetivo de demonstrar que software realiza as funções esperadas e revelar eventuais defeitos antes de sua utilização em ambiente real. Ele ajuda a identificar defeitos, melhorar a qualidade do produto final e garantir maior confiabilidade, segurança e desempenho para os usuários. Sem uma estratégia de testes bem definida, aumentam os riscos de erros na produção, retrabalho e insatisfação dos clientes.

Nos últimos anos, os Modelos de Linguagem de Grande Escala, (*Large Language Models,* LLM) transformaram significativamente a forma como as atividades de engenharia de software são realizadas (White et al., 2023). Sistemas que utilizam LLMs como o ChatGPT e Deep Seek, passaram a ser amplamente utilizados por desenvolvedores para gerar códigos, auxiliar na documentação, simular sistemas e até mesmo refatorar e testar software. Segundo White et al. (2013), LLMs demonstram potencial para auxiliar em diversas atividades da engenharia de software, graças à sua capacidade de entender linguagem natural e gerar artefatos de software. O que torna esses modelos tão acessíveis e poderosos é a possibilidade de interagir com eles por meio de prompts - instruções em linguagem natural que orientam o modelo a executar tarefas específicas. De acordo com White et al. (2013), explicam que os prompts podem ser compreendidos com uma forma de programação, utilizada para orientar os LLMs na execução de tarefas relativas à engenharia de software.

1.1 Justificativa

Nesse contexto, os LLMs, como ChatGPT, têm se mostrado promissores na automação e suporte de tarefas relacionadas aos testes de software, inclusive com desempenho superior à abordagem convencional. Isso inclui atividades de verificação e validação, cujo objetivo, conforme destaca Sommerville(2011), é garantir que o software esteja adequado ao uso pretendido, gerando confiança quanto a sua funcionalidade e conformidade com os requisitos definidos. Esses modelos são capazes de compreender requisitos, sugerir casos de testes, identificar ambiguidade e até simular o comportamento de sistemas.

Este trabalho se justifica pela lacuna existente no estado da arte quanto à aplicação prática de LLMs no teste de software, especialmente na geração e automação de casos de teste com critérios consolidados na literatura. Embora haja indícios de que essas ferramentas possam ampliar a cobertura, detectar falhas e reduzir o esforço humano, a maioria dos estudos ainda se concentra em análises conceituais ou exemplos pontuais, sem avaliações sistemáticas de sua efetividade. Nesse sentido, a condução de experimentos controlados permite mensurar de forma objetiva a qualidade dos artefatos gerados, a eficiência na detecção de falhas e o nível de automação alcançado, fornecendo evidências concretas que sustentem recomendações e boas práticas para o uso de LLMs na garantia da qualidade de software.

1.2 Objetivo

Este trabalho tem como propósito principal investigar o potencial de LLMs no apoio às atividades de teste de software. Para isso, foram definidos os objetivos a seguir, que orientam a condução da pesquisa e sustentam a construção de um referencial teórico e prático sobre o tema.

1.2.1 Objetivo Geral

Analisar a utilização de LLMs com apoio às atividades de teste de software, com foco em sua aplicação prática, nos benefícios oferecidos, nas limitações identificadas e nos impactos gerados na qualidade dos testes realizados. Incluindo a avaliação da capacidade das LLMs em projetar casos de testes aplicando critérios de cobertura e automatizar testes por meio de geração de scripts executáveis.

1.2.2 Objetivo Específico

- Identificar as atividades de teste de software que podem ser apoiadas por LLMs;
- Investigar os tipos de testes e artefatos que podem ser gerados, otimizados ou enriquecidos com o uso de LLMs;
- Estruturar boas práticas e estratégias para orientar a aplicação eficaz dos LLMs no processo de teste de software;
- Avaliar, por meio de experimentos, a qualidade dos artefatos de teste gerados com base em critérios como clareza, completude e utilidade.

1.3 Organização do Trabalho

A organização deste trabalho, além da introdução, está estruturada em sete capítulos. O capítulo 2 apresenta a fundamentação teórica que sustenta este estudo, abordando conceitos essenciais para a compreensão do tema. No capítulo 3, são apresentados e discutidos os trabalhos relacionados, com objetivo de contextualizar a pesquisa no estado da arte. O capítulo 4 descreve a metodologia adotada para o desenvolvimento do estudo. O capítulo 5 apresenta as diretrizes para o uso de LLMs em teste de software. O capítulo 6 por sua vez, apresenta os resultados do estudo, acompanhado de uma análise dos resultados obtidos. Por fim, o capítulo 7 apresenta as considerações finais e trabalhos futuros.

2 FUNDAMENTAÇÃO TEÓRICA

Neste capítulo serão discutidos alguns conceitos e termos importantes para o entendimento deste trabalho.

2.1 Teste de Software

O Teste de Software é uma das principais atividades do processo de desenvolvimento de sistemas, sendo um processo dinâmico de execução de programas com o objetivo de identificar falhas e verificar se o comportamento do sistema está de acordo com o esperado, o que torna fundamental para assegurar que o produto final atenda as expectativas dos usuários e as especificações técnicas estabelecidas. De acordo com Ammann e Offutt (2017), esse tipo de avaliação é uma das principais formas utilizadas pela indústria para validar o software durante o desenvolvimento.

Segundo Delamaro, Maldonado e Jino (2007), a construção de software, por ser uma atividade essencialmente humana, está sujeita a falhas, o que torna imprescindível a verificação do comportamento do sistema por meio de testes. Esses testes são conduzidos a partir de entregas conhecidas, com o objetivo de verificar se as saídas produzidas estão corretas, revelando assim, possíveis defeitos.

Além de ser uma atividade que contribui diretamente para a detecção de defeitos, o teste de software também exerce um papel estratégico na manutenção da confiabilidade dos sistemas. Conforme Aniche (2022), em um cenário de desenvolvimento contínuo e ágil, os testes automatizados permitem que as equipes avancem com segurança, evitando regressões e assegurando que o software continue funcionando conforme o esperado mesmo após várias modificações. Dessa forma, o teste não é apenas uma etapa final, mas uma prática integrada ao ciclo de vida do software desde as fases iniciais.

2.1.1 Conceitos Fundamentais

Nesta seção, serão abordados termos técnicos utilizados na literatura e na prática profissional, como defeito, erro, falha, teste, teste unitário, teste integração, teste de sistema e depuração (*debug*), caso de teste e critérios de cobertura, os quais são fundamentais para a comunicação técnica entre as equipes de desenvolvimento, qualidade e gestão de projetos.

O termo defeito refere-se a uma instrução ou decisão incorreta no código do programa. De acordo com Ammann e Offutt (2017), o defeito é um erro estático, introduzido por um engano humano durante a construção do software. Ele pode estar relacionado a algoritmos

errados, estruturas mal definidas ou falhas na lógica de negócio. Conforme o autor Aniche (2022) destaca que a identificação e correção de defeitos são objetivos do processo de teste.

O erro, conforme definido por Ammann e Offutt (2017), é um estado interno incorreto do programa durante sua execução. Ele é resultado da ativação de um defeito e pode levar a comportamentos inesperados. No contexto do teste, a detecção de um erro é o primeiro indício de que o sistema pode estar em desacordo com a especificação. O autor Aniche (2022) reforça que o erro não necessariamente se manifesta ao usuário, mas compromete a lógica interna do sistema.

Segundo Ammann e Offutt (2017), uma falha é observada quando o resultado real difere do esperado durante a execução de um caso de teste. É nesse momento que se evidencia a existência de um defeito no código, sendo a falha a manifestação mais perceptível de um problema interno no software.

Segundo Delamaro, Maldonado e Jino (2007), o teste de software é uma atividade complexa que busca evidenciar a presença de defeitos e não provar a ausência deles. Ainda de acordo com o autor, é uma fase crucial para a melhoria da qualidade do software, embora possa representar um custo significativo no desenvolvimento. A atividade de teste é dividida em etapas bem definidas, que incluem planejamento, projeto de casos de teste, execução e análise dos resultados.

De acordo com Ammann e Offutt (2017) eles explicam que existem diferentes formas de realizar o teste, que buscam selecionar um subconjunto de dados de entrada que tenham alta probabilidade de revelar defeitos. Para os autores, os critérios de testes são as regras usadas para identificar subdomínios e selecionar casos de testes, podendo ser funcionais, estruturais ou baseado em defeitos. Idealmente, o teste deveria ser exaustivo, cobrindo todas as entradas possíveis, mas os autores ressaltam que isso geralmente é inviável devido a grande quantidade de combinações

As atividades de teste de software são tipicamente divididas em fases distintas para lidar com a complexidade e os diferentes níveis de abstração dos erros, segundo Delamaro, Maldonado e Jino (2007).

O teste de unidade, conforme Delamaro, Maldonado e Jino (2007) e Aniche (2022) discutem que esta fase foca nas menores unidades de um programa como funções, procedimentos, métodos ou classes. Os autores continuam, que o objetivo é identificar erros relacionados a algoritmos incorretos, implementações falhas ou falhas de programação em componentes individuais. Segundo Aniche (2022), testes de unidade são geralmente rápidos e fáceis de controlar, permitindo feedback constante ao desenvolvedor.

Testes de integração são realizados após o teste de unidade, o teste enfatiza a construção da estrutura do sistema, segundo Delamaro, Maldonado e Jino (2007). O autor destaca que o foco é verificar se a interação entre as diversas partes do software funciona corretamente e não gera novos erros. De acordo com os autores, esse tipo de teste tende a ser executado pela equipe de desenvolvimento devido a necessidade de conhecimento sobre estruturas internas e interações do sistema. De acordo com Aniche (2022) que testes são mais complexos de configurar e mais lentos que os testes de unidade, envolvendo frequentemente componentes externos como banco de dados e serviços web.

Teste de sistema, o autor Aniche (2022) afirma que os teste de sistema inicia-se quando o sistema está completo, com todas as suas partes integradas. Conforme Delamaro, Maldonado e Jino (2007), o objetivo é verificar se as funcionalidades especificadas nos documentos de requisitos estão todas corretamente implementadas. Os autores mencionam que muitas organizações adotam a estratégia de designar uma equipe independente para realizar o teste de sistema.

De acordo com Delamaro, Maldonado e Jino (2007) definem depuração ou *debugging*, como a atividade de localizar e remover defeitos (falhas) no software. Diferentemente do teste, cujo objetivo é mostrar que o software não funciona como o esperado, autores explicam que a depuração deve localizar as causas que levaram os software a falhar e indicar como ele deve ser corrigido. E os autores ainda afirmam que, historicamente, a depuração baseou-se principalmente na habilidade e experiência das pessoas encarregadas dessa tarefa, sem que fossem utilizadas técnicas sistemáticas. É importante notar que, em muitos casos, decidir se dois programas são equivalentes é um problema indecidível, mas Delamaro, Maldonado e Jino (2007) apontam que heurísticas e a intervenção humana podem ajudar a superá-lo. A automatização é crucial para a depuração, pois a demanda computacional é alta para tarefas como a geração e execução de mutantes e a comparação de resultados.

Um caso de teste representa uma situação específica a ser testada no software, contendo, geralmente, uma entrada específica acompanhada do comportamento esperado, utilizado para verificar se o sistema se comporta de maneira correta. De acordo com Aniche (2022), cada caso de teste é um artefato que deve ser elaborado com clareza e precisão, contendo os dados de entrada, a pré-condição, ações a ser executada e o resultado esperado. Segundo Delamaro, Maldonado e Jino (2007), o conjunto de todos os casos de testes aplicados em uma execução forma o chamado conjunto de testes.

Conforme Ammann e Offutt (2017), os critérios de cobertura são fórmulas destinadas a estabelecer os requisitos de testes que serão efetivamente cobertos pelos casos de testes. De

acordo com Delamaro, Maldonado e Jino (2007), esses critérios ajudam a definir subconjuntos representativos do domínio de entrada e da estrutura do programa, guiando a elaboração de testes mais eficazes. Esses critérios podem ser classificados de acordo com os artefatos utilizados para gerar casos de teste. No teste funcional, por exemplo, a base é a especificação do software, enquanto no teste estrutural, a referência é o código-fonte. Nesse sentido Aniche (2022) ressalta que os critérios de cobertura podem ser aplicáveis tanto com testes funcionais (como classes de equivalência e valor limite) quanto em testes estruturais (como cobertura de linha e decisão), constituindo ferramentas importantes para a medição da qualidade de testagem.

2.1.2 Atividades do Processo de Teste de Software

O processo de teste de software é composto por um conjunto estruturado de atividades que, de forma complementar, buscam verificar se o sistema funciona conforme especificado e identificar possíveis falhas antes que o produto seja entregue ao usuário final. Essas atividades não ocorrem de maneira isolada: elas se interrelacionam e se retroalimentam continuamente, formando um ciclo de melhoria da qualidade. Conforme apontam Delamaro, Maldonado e Jino (2007), as principais etapas envolvem projeto de testes, a execução, a avaliação dos resultados e a automação. Cada uma delas desempenham um papel importante para a eficiência dos testes e requer atenção especial em sua aplicação.

A fase de projeto de teste é o momento em que os casos de teste são elaborados com base em critérios previamente definidos. Segundo Aniche (2022), essa é uma das atividades mais importantes no processo, pois a escolha dos dados de entrada e dos resultados esperados influencia diretamente a capacidade do teste de encontrar falhas. O autor ressalta que testes bem projetados reduzem redundâncias, aumentam a legibilidade e melhoram a manutenção dos artefatos de testes ao longo do tempo. Além disso, como destacado por Delamaro, Maldonado e Jino (2007). o projeto deve considerar tanto os aspectos funcionais quanto estruturais do sistema, buscando equilíbrio entre cobertura e custo.

Após o projeto, os testes são executados no sistema de teste. Nessa etapa, o objetivo é aplicar os casos de testes definidos e observar o comportamento do programa em tempo de execução. Segundo o guia prático de testes automatizados por Aniche (2022), a execução pode ser feita de forma manual ou automatizada, dependendo dos recursos disponíveis e do contexto do projeto. Durante essa etapa, registra-se a ocorrência de falhas, exceções ou comportamentos inesperados, sendo essencial manter a rastreabilidade entre os testes e os requisitos que eles cobrem.

A automação tem ganhado papel de destaque nos processos modernos de desenvolvimento, especialmente com o uso de metodologias ágeis e práticas de integração contínua. Conforme destaca Aniche (2022), a automação visa reduzir o esforço manual e permitir a reexecução frequentes de teste de maior confiabilidade, e complementa que a automação facilita a identificação de regressões, aumenta a cobertura de testes e melhora a produtividade da equipe. No entanto, ele alerta que nem todos os testes devem ser automatizados: é importante avaliar o custo de manutenção, a estabilidade do código e a complexidade envolvida.

Uma vez executados os testes, os resultados obtidos devem ser comparados com os resultados esperados previamente definidos. Essa avaliação tem como objetivo identificar se houve divergências, que indicariam a presença de falhas. De acordo com Delamaro, Maldonado e Jino (2007), essa análise precisa ser criteriosa, pois nem toda falha observada representa necessariamente um erro no software, ela pode estar relacionada a um especificação incompleta ou a um caso de teste mal formulado. Para Aniche (2022), essa etapa é fundamental para retroalimentar o processo de testes, refinando os casos futuros e ajustando os critérios utilizados.

2.1.3 Teste Estrutural

O teste estrutural, também conhecido como teste de caixa-branca, tem como foco a verificação de comportamento interno do sistema. Para isso, exige que o testador possua conhecimento detalhado sobre o código-fonte, de forma a garantir que todos os caminhos, decisões e instruções relevantes do programa sejam exercidos durante a execução dos testes. Segundo Delamaro, Maldonado e Jino (2007), essa abordagem é particularmente eficaz para identificar falhas lógicas, instruções não executadas e caminhos de execução pouco explorados. De acordo com Aniche (2022) destaca ainda que, ao aplicar critérios estruturais, é possível quantificar a cobertura alcançada e aprimorar a eficiência dos testes.

O teste de cobertura de linha é um dos critérios mais básicos e intuitivos de teste estrutural. Seu objetivo é verificar se todas as linhas de código do sistema foram executadas ao menos uma vez durante os testes. Embora simples, essa técnica permite identificar instruções não executadas. Segundo o autor Aniche (2022), essa abordagem é útil em fases iniciais do desenvolvimento, embora não assegure que todas as decisões e fluxos de controle tenham sido devidamente exploradas. Os autores Delamaro, Maldonado e Jino (2007) complementam que a cobertura de linha deveria ser acompanhada de critérios mais exigentes para que a testagem seja mais eficaz na identificação de defeitos sutis.

O teste de cobertura de ramificação amplia a profundidade do teste ao exigir que todas as decisões lógicas do programa sejam avaliadas tanto no caminho verdadeiro quanto no falso. Essa técnica é particularmente importante em estruturas condicionais, como *if, switch* e *laços while*. Conforme Delamaro, Maldonado e Jino (2007), esse critério proporciona uma verificação mais abrangente do comportamento lógico do sistema. Já Aniche (2022), aponta que a cobertura de ramificação é um ponto de equilíbrio entre simplicidade e profundidade, sendo amplamente adotado por times que desejam garantir a confiabilidade sem comprometer a produtividade dos testes.

O teste de cobertura de condições visa garantir que cada condição dentro de uma decisão seja avaliada isoladamente como verdadeira e falsa. Por exemplo, em uma decisão como if (idade > 18 && temHabilitacao == true), é necessário testar individualmente a condição (idade > 18) para verdadeiro e falso, e também a condição (temHabilitacao == true) para verdadeiro e falso, independente do resultado global da decisão. Isso é relevante porque, mesmo que a decisão completa já tenha sido exercitada, suas condições internas podem não ter sido exploradas adequadamente. Isso significa que, mesmo em uma estrutura de decisão que já tenha sido exercitada como um todo, suas condições internas podem não ter sido completamente exploradas. Conforme Delamaro, Maldonado e Jino (2007), essa técnica é essencial quando as decisões envolvem múltiplas expressões lógicas combinadas com operadores com && e ||. O autor Aniche (2022), destaca que, ao aplicar esse critério, o testador amplia a capacidade de encontrar falhas sutis que dependem de condições específicas e complexas no código.

2.1.4 Teste Funcional

O teste funcional, também conhecido como teste de caixa-preta, baseia-se exclusivamente na especificação dos requisitos, sem considerar a estrutura interna do sistema. Segundo Delamaro, Maldonado e Jino (2007), esse tipo de teste busca validar se o sistema responde corretamente as entradas fornecidas e produz as saídas esperadas, conforme definido pela documentação. O autor Aniche (2022) destaca que o teste funcional é particularmente eficaz para identificar falhas na lógica de negócio, sendo útil mesmo quando o código-fonte ainda não está disponível. O autor Aniche (2022) observa que essas técnicas funcionais são adequadas para todos os níveis de teste, incluindo testes de unidade, integração e sistema.

A técnica de teste de particionamento em classe de equivalência divide o domínio de entrada do sistema em subconjuntos, com base na premissa de que todos os elementos de uma mesma classe devem ser processados da mesma maneira. Conforme Delamaro, Maldonado e

Jino (2007), a ideia é testar apenas um representante de cada classe, reduzindo significativamente o número de casos de testes necessários sem comprometer a cobertura. O autor Aniche (2022) aponta que essa abordagem é especialmente útil quando se deseja explorar múltiplas entradas válidas e inválidas, garantindo a representatividade dos testes com economia de esforço.

A análise de valor limite é uma extensão do particionamento de classes, voltada para testar os extremos de cada faixa de entrada. Segundo Delamaro, Maldonado e Jino (2007), a experiência prática mostra que valores próximos aos limites são mais propensos a falhas, tornando esse tipo de teste essencial para o sistema. O autor Aniche (2022) reforça que o teste nos pontos de fronteiras é fundamental em sistemas que operam com intervalos numéricos, datas, limites de tamanho ou outros parâmetros críticos, o autor recomenda essa abordagem para detectar erros de arredondamento ou comparação mal implementadas.

Na prática, a combinação das duas técnicas anteriores é bastante recomendada. De acordo com Delamaro, Maldonado e Jino (2007), essa unificação aumenta a probabilidade de detectar as falhas ao testar representações das classes e seus respectivos valores. O autor Aniche (2022) sugere, por exemplo, testar o primeiro, o último e um valor intermediário de cada classe de equivalência, o que ajuda a verificar tanto o comportamento esperado quanto possíveis exceções, o autor ainda ressalta que essa abordagem híbrida é comum em testes de API's, validações de formulários e componentes críticos do sistema.

2.1.5 Testes Automatizados.

Os autores Ammann e Offutt (2017) definem o teste automatizado com o uso de software para controlar a execução dos testes, comparar os resultados obtidos com os resultados esperados, configurar condições prévias e gerenciar atividades como controle e geração de relatório. Eles ressaltam que, como o teste de software pode ser muito caro e demandar muito esforço humano, um objetivo interessante é automatizar o máximo possível. Essa prática não apenas reduz custos, mas também minimiza erros humanos e facilita a realização de testes de regressão, permitindo sua repetição com apenas um clique.

2.2 Modelos De Linguagem De Grande Escala (LLMs)

Os Modelos de Linguagem de Grande Escala, conhecidos como LLMs (do inglês *Large Language Models*), são modelos de processamento de linguagem natural desenvolvidos com base em avanços recentes da inteligência artificial (IA). De acordo com Belzner, Gabor e Wirsing (2023), esses modelos incluem sistemas como o GPT da OpenAI e o Gemini da

Google, reconhecidos por sua capacidade de atuar de forma conversacional e orientada por intenção, auxiliando o desenvolvimento de software em diversas tarefas. Sua arquitetura, geralmente baseada em transformadores, permite que modelos capturem relações semânticas e contextuais complexas em grandes volumes.

Na prática, os LLMs são aplicados com assistentes inteligentes em tarefas como análise de requisitos, sugestão de nomes de métodos, geração de casos de testes, análise de blogs e criação de APIs. De acordo com White et al. (2023), o uso de padrões de prompts específicos permite guiar a interação com esses modelos de forma eficiente, extraindo deles respostas úteis e contextualizadas para tarefas técnicas. A interação ocorre por meio de comandos de linguagem natural, os quais são interpretados pelo modelo que então gera respostas que podem incluir trechos de código, documentação ou planos de teste.

A aplicação prática dos LLMs ocorre principalmente por meio de interfaces baseadas em prompts. Em vez de comandos rígidos, os desenvolvedores descrevem suas intenções em linguagem natural, e o modelo responde com a solução mais provável. Essa abordagem é explorada, por exemplo, por Liu K. et al. (2024), que propuseram o uso de LLMs para geração automatizada de teste com alta cobertura funcional, especialmente voltados à detecção de bugs complexos. Nesse cenário, os modelos atuam como especialistas, produzindo entradas e oráculos com base nas especificações dos sistemas analisados.

Outro diferencial relevante dos LLMs é sua capacidade de adaptação a diferentes domínios, sem a necessidade de ajustes estruturais no modelo. Essa característica é particularmente útil na engenharia de software, onde os modelos podem compreender descrições em linguagem natural e contribuir em atividades como definição de requisitos, planejamento de arquiteturas e revisões de código. Como discutido por Belzner, Gabor e Wirsing (2023), os LLMs operam de forma contextual, ajustando suas respostas com base no domínio técnico e na natureza da tarefa, o que os torna úteis em ambientes dinâmicos e multidisciplinares.

Adicionalmente, os LLMs demonstram competências em interações iterativas, sendo capazes de refinar suas respostas ao longo da conversa com o usuário. Como analisado por Belzner, Gabor e Wirsing (2023), essa flexibilidade permite que os modelos adaptem seu comportamento com base no histórico da interação no contexto da tarefa, possibilitando sua atuação em diversas fases do desenvolvimento so software. Essa capacidade contínua de adaptações permite que os LLMs apoiem desde a elicitação de requisitos até a geração de testes e cenários de uso, mesmo diante de descrições ambíguas ou de alto nível, que durante o

processo de pedidos de informações pode ser usado alguma padrão de prompt para melhor eficiente da LLM.

2.2.1 Engenharia de Prompt

A engenharia de prompts é a prática de construir instruções cuidadosamente estruturadas para orientar o comportamento de LLMs. Essa abordagem tem ganhado destaque por sua capacidade de influenciar diretamente a qualidade, relevância e utilidade das respostas geradas. Segundo White et al. (2023), os prompts funcionam como uma forma de "programação livre", permitindo controlar o estilo, o conteúdo e até o raciocínio do modelo, especialmente em contextos técnicos como a engenharia de software. Para garantir essa prática, os autores propõem um catálogo com 16 padrões de prompts organizados em grupos conceituais.

Entre os padrões voltados à estruturação da entrada e personalizações das interações, destacam-se o Meta Language Creation, que define uma linguagem artificial para comunicação mais clara com o modelo, e o Persona, no qual o LLM assume o papel de um profissional específico, moldando o tom e o vocabulário das respostas. Já o Flipped Interaction inverte a lógica da conversa, fazendo com que o modelo formule perguntas ao usuário. O padrão Question Refinement permite que a LLM reformule a pergunta original para torná-la mais clara ou objetiva, enquanto o Alternative Approaches incentiva a geração de múltiplas soluções para um mesmo problema. O Cognitive Verifier divide questões complexas em partes menores antes de apresentar uma resposta final, e o Fact Check List faz com que o modelo declare os fatores em que se baseou, aumentando a transparência da resposta.

Outros padrões se concentram na estrutura de saída. O Template fornece um modelo fixo para respostas, garantindo uniformidade, enquanto o Output Automater instrui o modelo a gerar comandos ou códigos automatizáveis. O Infinite Generation permite que a LLM continue produzindo conteúdo sem interrupção, útil em tarefas criativas. Já o Visualization Generator pede ao modelo que descreve visualmente dados que podem ser transformados em gráficos ou imagens. Padrões como Game Play simulam interações lúdicas para aprendizagem, enquanto o Reflection faz com que o modelo revise sua própria resposta. O Refusal Breaker reformula perguntas rejeitadas inicialmente pelo LLM, e o Context Manager ajuda a manter a coerência em interações mais longas. Por fim, o Recipe organiza o raciocínio do modelo em etapas sequenciais, mesmo quando o usuário fornece informações incompletas.

3 TRABALHOS RELACIONADOS

Neste capítulo, são descritos alguns trabalhos relacionados ao uso de LLMs, com ênfase na geração de teste de software e engenharia de prompt. Os estudos selecionados abordam diferentes abordagens, técnicas e ferramentas aplicadas em contextos práticos, além de destacarem benefícios e limitações dessas soluções. A análise desses trabalhos permite estabelecer conexões com a proposta deste trabalho, contribuindo para a contextualizar sua relevância e fundamentar sua construção teórica e prática.

3.1 Make LLM a Testing Expert: Bringing Human-like Interaction to Mobile GUI Testing via Functionality-aware Decisions.

Liu Z. et al. (2024) propõe uma abordagem inovadora para testes automatizados de interfaces gráficas móveis (GUI) por meio do sistema GPTDroid, que transforma o problema de testes em uma tarefa de perguntas e respostas (QA) com um modelo de linguagem de grande escala (LLM), como o ChatGPT. Diferentemente de abordagens anteriores baseadas em regras ou aprendizado de máquina, o GPTDroid interage com aplicativo sob teste de forma similar a um testador humano, interpretando a estrutura da interface e gerando comandos de ação com bases em uma história de interações.

O diferencial da proposta está na introdução de um mecanismo de memória consciente de funcionalidade, que permite ao LLM reter informações sobre o progresso dos testes e raciocinar sobre as funcionalidades exploradas ao longo se sessões de testes de GUI: o dilema local (perda de contexto interações longas) e o dilema de baixo nível (foco) excessivo em elementos isolados da interface, ignorando fluxos mais amplos.

Nos experimentos conduzidos com 93 aplicativos Android populares da Google Play Store, o GPTDroid obteve 75% de cobertura de atividades e 66% de cobertura de código, superando em até 32% as ferramentas automatizadas existentes. Além disso, foram detectados 53 novos bugs em aplicativos reais, das quais 35 foram confirmados e corrigidos por desenvolvedores, demonstrando a aplicabilidade prática da abordagem.

Entre as principais contribuições do estudo estão, a formulação de teste de GUI como tarefa de QA, a implementação de uma memória funcional para raciocínio contextual e global, a geração automática de comandos executáveis a partir de interpretações da interface, a avaliação em cenários reais, com resultados superiores a ferramentas baseadas em aprendizado de máquina, regras ou modelos de estados.

3.2 LLM-Powered Test Case Generation for Detecting Tricky Bugs.

Liu K. et al. (2024) abordam um dos principais desafios em testes de software: a geração automatizada de casos de testes de detectar bugs sutis em programas plausivelmente corretos - ou seja, sistemas que passam em todos os testes existentes, mas ainda contém falhas funcionais não detectadas. Para lidar com essa lacuna, os autores propõem a abordagem AID (AI-driven Differential testing), que combina LLMs com testes diferenciais para gerar entradas e oráculos de testes mais eficazes.

A arquitetura do AID é composta por três etapas principais: geração de variáveis do programa, a partir do código sob teste (PUT) e sua especificação, com filtragem por testes existentes para garantir qualidade mínima; geração de entradas de testes por meio de geradores de código (como scripts em Python), em vez de entradas diretas, o que aumenta a validade e reduz erros de entrada; testes diferenciais priorizando diversidade, onde as saídas divergentes entre variantes indicam potenciais defeitos, permitindo a criação de oráculos confiáveis.

Uma das principais contribuições do trabalho é a substituição da tradicional votação por maioria nos testes diferenciais por uma estratégia baseada em diversidade de saídas, que melhora a detecção de bugs complexos. Além disso, o uso de geradores de entrada evita que o modelo produza entradas inválidas, problema comum em abordagens LLMs diretas.

Os experimentos realizados nos conjuntos Tricky bugs (com programas em C + + e Python oriundos de competições de programas) e EvalPlus (códigos gerados por IA) demonstraram ganhos expressivos. O AID superou métodos de ponta como DPP e APR, atingindo melhorias de 1,80x em recall, 2,65x em precisão e 1,66x em F1 score. A precisão variou entre 69,3% e 85,1%, destacando-se como uma solução viável para aplicação em ambientes de desenvolvimento real.

Por fim, estudos e ablação realizados pelos autores evidenciam que cada componente do AID - desde a geração guiada de variantes até o teste diferencial orientado à diversidade - contribuiu de forma significativa para seu desempenho superior. A abordagem representa um avanço relevante na utilização de LLMs para geração de testes, especialmente voltada à identificação de bugs que escapam de suites tradicionais.

3.3 ChatGPT prompt patterns for improving code quality, refactoring, requirements elicitation, and software design.

White et al. (2023) propõem uma abordagem sistemática para aumentar a efetividade de interações com LLMs , como o ChatGPT, por meio de padrões de prompts especificamente a engenharia de software. O artigo introduz um catálogo de 13 padrões organizados em quatro categorias principais: Elicitação de Requisitos, Design e Simulação, Qualidade de Código e Refatoração. Esses padrões são inspirados em princípios similares aos dos padrões de software, oferecendo soluções reutilizáveis para problemas recorrentes na comunicação com LLMs.

Cada padrão é descrito com uma estrutura padronizada, incluindo: nome, intenção, contexto, motivação, estrutura, exemplo de implementação e consequências. A proposta central é que esses padrões funcionem como programação leve por meio da linguagem natural, permitindo configurar o comportamento do modelo em diferentes contextos técnicos. Por exemplo, o padrão Requirements Simulator permite simular interações com o sistema a partir de requisitos, revelando lacunas e permitindo a melhoria iterativa das especificações. Já o padrão API Generator auxilia na transformação de descrições funcionais em especificações REST(como OpenAPI), promovendo maior formalização antecipada no ciclo de desenvolvimento.

Entre os padrões voltados a qualidade de código, destacam-se o Code Clustering, que orienta a separação de responsabilidades no código (como lógica de negócio e acesso a dados), e o Intermediate Abstraction, que recomenda a inserção de camadas intermediárias para reduzir o acoplamento com bibliotecas externas. Esses padrões visam não apenas melhorar o código gerado por LLMs, mas também facilita sua manutenção e escalabilidade. Outros padrões como Principled Code e Hidden Assumption auxiliam na produção de código aderente a boas práticas e na identificação de premissas ocultas, respectivamente.

O trabalho destaca ainda que os LLMs tendem a apresentar limitações quando utilizados sem orientação adequada, como a geração de código monolítico ou a omissão de requisitos essenciais. A aplicação consciente dos padrões descritos permite potencializar a capacidade dos modelos, reduzindo erros, melhorando a cobertura de casos e agilizando atividades críticas no desenvolvimento de software. Essa abordagem também se mostra eficaz quando combinada com outras técnicas, como geração de exemplos few-shot e simulação iterativa.

Em síntese, o catálogo de padrões de prompt apresentado por White et al.(2023) contribui significativamente para a consolidação da engenharia de prompt como uma

competência essencial no uso de LLMs na engenharia de software. Ao formalizar boas práticas de interações com esses modelos, o artigo oferece uma referencial prático e conceitual que pode ser adaptado a diferentes estágios do ciclo de vida do software, favorecendo maior produtividade e qualidade nas soluções geradas por IA.

3.4 A prompt pattern catalog to enhance prompt engineering with ChatGPT.

O artigo de White et al.(2023) propõe um catálogo abrangente de padrões de prompt com o objetivo de fortalecer a prática da engenharia de prompt aplicada a LLMs como o ChatGPT. O trabalho reconhece que, embora esses modelos possuam grande capacidade de geração textual, a eficiência de duas respostas está diretamente relacionada à forma como são instruídas. Para isso, os autores introduzem uma estrutura inspirada nos padrões de projeto de software, permitindo criar, documentar e reutilizar prompts de maneira sistemática e eficaz. Essa abordagem busca padronizar a construção de instruções, facilitando a replicação de boas práticas em diferentes domínios de aplicação.

O catálogo apresentado inclui 16 padrões de prompt, cada um descrito com base em uma estrutura fixa que abrange nome, intenção, motivação, contexto de uso, formato de entrada, exemplo prático e consequências do uso. Esses padrões foram desenvolvidos para resolver problemas recorrentes na interação com LLMs, como a geração de saídas genéricas, incoerentes ou irrelevantes.Entre os exemplos citados, destacam-se o Meta Language Creation, voltado a criação de linguagens específicas para comunicação mais precisa, o Output Automater, que guia o modelo de produção de saídas diretamente executáveis; e o Persona, que molda o estilo da resposta ao fazer o modelo assumir o papel de um especialista.

Além da descrição individual dos padrões, os autores demonstram como a combinação de diferentes padrões pode ampliar ainda mais a eficiência dos prompts. Essa abordagem composicional permite criar instruções mais complexas, adaptadas a contextos específicos e objetivos variados. Por exemplo, um prompt pode combinar a estrutura do Question Refinement com a orientação prática do Fact Check List para gerar respostas mais claras, verificáveis e úteis. Ao final, o estudo reforça a importância da engenharia de prompt como disciplina emergente e destaca possíveis caminhos para pesquisas futuras, principalmente no desenvolvimento de ferramentas automatizadas para apoiar a aplicação desses padrões em escala.

3.5 Large Language Model Assisted Software Engineering: Prospects, Challenges, and a Case Study Preprint.

O artigo Belzner, Gabor e Wirsing (2023), examina as possibilidades de aplicação de LLMs, como o ChatGPT, ao longo do ciclo de vida da engenharia de software. Os autores propõem um modelo conceitual no qual os LLMs atuam como copilotos inteligentes, auxiliando engenheiros humanos em tarefas como elicitação de requisitos, design de sistemas, geração de código, documentação, testes e manutenção. A principal premissa do estudo é que as LLMs podem ampliar a produtividade e a qualidade dos processos de desenvolvimento ao oferecer suporte contextualizado e orientado por intenção, especialmente em tarefas que envolvem ambiguidade ou requisitos mal definidos.

Uma das contribuições centrais do artigo é a ênfase na interação interativa entre engenharia e modelos, defendendo o uso dos LLMs em ciclos de refinamento, nos quais a resposta do modelo é ajustada com base no feedback humano. Essa abordagem se mostrou eficaz no estudo de caso apresentado pelos autores, que envolveu a aplicação de LLMs na análise de requisitos e na geração de testes para um sistema de transporte relevante e bem contextualizado, embora a validação humana ainda seja necessária para garantir precisão e adequação.

O estudo também discute os desafios associados ao uso de LLMs na engenharia de software, destacando o risco de respostas imprecisas (alucinações), a dificuldade de integração com ferramentas e pipelines existentes, e a necessidade de maior controle sobre o comportamento do modelo. Os autores concluem que o avanço na área requer o desenvolvimento de técnicas de engenharia de prompt mais refinadas, bem como a consideração de aspectos éticos, como responsabilidade pelas decisões baseadas em saídas geradas por IA. O trabalho contribui para consolidar o papel dos LLMs como ferramentas promissoras, mas que exigem uso crítico e supervisionado no contexto de engenharia de software.

3.6 Discussões

Os trabalhos revisados apresentam abordagem distintas e complementares para a aplicação de LLMs na engenharia de software, com ênfase e melhoria da qualidade do desenvolvimento. Cada proposta explora dimensões diferentes do problema, desde soluções práticas para automação de testes até frameworks conceituais que estruturam a interação com os modelos, revelando o potencial das LLMs para transformar práticas tradicionais de testes e desenvolvimento.

O estudo de Liu Z. et al. (2024) propõe a ferramenta GPTDroid, que transforma o teste de interfaces gráficas móveis (GUI) em uma tarefa de perguntas e respostas, simulando o raciocínio humano. O diferencial da abordagem está na utilização de uma memória funcional que preserva o contexto ao longo das sessões, mitigando problemas comuns como perda de histórico ou excesso de foco em elementos isolados, um desafio comum em testes exploratórios automatizados. Essa característica, aliada à capacidade do modelo de interpretar a estrutura da interface e gerar comandos de ação de forma dinâmica, resultou em alta cobertura de código e descoberta de bugs reais em experimentos com aplicativos populares. Esses resultados evidenciam não apenas a aplicabilidade de prática da solução, mas também seu potencial para reduzir esforço humano em tarefas repetitivas e aumentar a efetividade dos testes em ambientes móveis, onde a diversidade de dispositivos e iterações eleva a complexidade.

Por sua vez, o trabalho de Liu K. et al. (2024) concentra-se na geração de casos de teste para identificação de bugs sutis em programas aparentemente corretos, problema notoriamente difícil para métodos convencionais. A proposta AID combina LLMs com testes diferenciais orientados à diversidade de saídas, estratégia que supera abordagens tradicionais baseadas em votação por maioria. Além disso, a integração com geração de entradas via scripts reduz inconsistências comuns em métodos baseados exclusivamente em instruções textuais, aumentando a robustez dos testes, reforçando a eficiência da técnica na detecção de falhas complexas, especialmente em software com múltiplas variações de comportamento. Essa pesquisa é relevante porque evidencia que LLMs podem atuar não apenas como geradores passivos de dados, mas como componentes ativos em estratégias híbridas de verificação.

Já o trabalho de White et al. (2023) segue uma linha metodológica distinta, ao introduzir um catálogo de padrões de prompt direcionados à engenharia de software. Esses padrões sistematizam boas práticas para orientar a interação com LLMs em atividades como elicitação de requisitos, design, refatoração e melhoria da qualidade do código. Ao estruturar a comunicação com modelos, a proposta busca mitigar limitações típicas, como incoerência nas respostas, falta de contextualização e geração de saídas genéricas, permitindo maior controle e previsibilidade nos resultados. Essa contribuição é estratégica porque estabelece uma base metodológica para a engenharia de prompt, prática cada vez mais reconhecida como essencial para maximizar a eficiência das LLMs no desenvolvimento de software.

Em síntese, enquanto os estudos de Liu Z. et al. (2024) e Liu K. et al. (2024) exploram aplicações práticas para automação de testes, com enfoques distintos - GUI e detecção de

bugs sutis, White et al. (2023) oferece um suporte conceitual que potencializa a eficiência dessas e de outras aplicações prévias. Juntas, essas abordagens evidenciam que o uso de LLMs na engenharia de software vai além de simples geração de textual, abrangendo desde estratégias específicas para automação até a definição de frameworks metodológicos que aprimoram a comunicação entre humanos e modelos.

Apesar dos avanços, os trabalhos analisados também apontam desafios e lacunas importantes. Em primeiro lugar, a maioria das propostas ainda está restrita a cenários experimentais ou ambientes controlados, o que limita a compreensão da escalabilidade em sistemas de grande porte. Além disso, há dependência significativa da qualidade dos prompts e da interpretação correta do contexto, fatores que impactam diretamente na consistência dos resultados. Outro ponto crítico é a variabilidade inerente aos LLMs, que pode gerar respostas diferentes para os mesmos comandos, prejudicando a reprodutibilidade — questão não resolvida por nenhuma das abordagens discutidas. Por fim, falta uma análise aprofundada sobre custos computacionais e riscos associados à integração em pipelines contínuos de desenvolvimento, aspectos fundamentais para viabilizar a adoção industrial.

Quadro 1 - Comparação dos trabalhos relacionados

Trabalho	Objetivo	Metodologia	Resultados	Limitações
Liu Z. et al. (2024) – GPTDroid	Teste automatizado de GUIs móveis com LLMs.	Q&A simulando raciocínio humano com memória funcional persistente	Alta cobertura de código; bugs reais encontrados.	Escopo restrito a GUIs móveis.
Liu K. et al. (2024) – AID	Detecção de bugs sutis em programas corretos.	Teste diferencial + geração de entradas por scripts.	Maior recall e precisão na detecção de falhas.	Não avalia cobertura estrutural; foco em programas específicos.
White et al. (2023)	Padrões de *prompt engineering* para engenharia de software.	Catálogo de boas práticas para orientar prompts.	Maior previsibilidade e coerência das saídas.	Não foca em automação de testes.

Fonte: Autoria própria(2025

4 METODOLOGIA

A metodologia deste trabalho foi estruturada para possibilitar uma investigação clara e organizada sobre o uso de LLMs no apoio às atividades de teste de software. O desenvolvimento da pesquisa será conduzido por meio das etapas sequenciais, que permitem alinhar os objetivos propostos com a execução prática e a análise dos resultados obtidos.

Essa abordagem metodologia será operacionalizada por meio de quatro etapas principais. A primeira consiste na revisão do estado da arte, com o propósito de mapear as principais contribuições, abordagens, desafios e lacunas identificadas na literatura sobre a aplicação de LLMs no teste de software.

A segunda etapa compreende a proposição de diretrizes e boas práticas para o uso de LLMs em atividades do Teste de Software. Este será apresentado na forma de um guia que foi desenvolvido com base nos principais achados da revisão da literatura e nas observações obtidas nas interações com LLMs. Este guia visa sistematizar estratégias de construção de prompts e critérios de avaliação, com o intuito de auxiliar na geração de testes mais eficazes.

Na terceira etapa, será conduzido um experimento de avaliação prático, no qual as tarefas apresentadas no guia serão aplicadas para gerar testes para programas extraídos de exemplos da literatura. Para estes experimentos, foi utilizada a versão 2.5 do Gemini Pro, um LLM desenvolvido pela Google e disponibilizado gratuitamente para estudantes por meio de assinatura educacional com e-mail institucional. Atualmente, usuários sem esse benefício têm acesso apenas à versão gratuita 1.5 do modelo. Esse experimento permitirá avaliar se os artefatos gerados pelo Gemini Pro apresentam qualidade equivalente aos produzidos manualmente, verificando a qualidade e a consistência dos testes gerados.

Por fim, a quarta etapa será dedicada à análise dos resultados dos experimentos. Essa análise se dará de forma qualitativa, a partir de uma avaliação da qualidade dos artefatos gerados, e quantitativa, a partir da extração de métricas obtidas a partir da execução dos testes gerados pelo LLM, utilizando as ferramentas JaCoCo[1] e PIT[2].

Com o JaCoCo, coletamos métricas de cobertura de instruções e cobertura de ramificações. A cobertura de instruções é a unidade mínima analisada pelo JaCoCo e corresponde a instruções individuais de Bytecodes Java. Essa métrica indica a quantidade de código efetivamente executado ou não durante os testes, mesmo na ausência de informações de depuração nos arquivos de classe. A cobertura de ramificações, por sua vez, calcula a cobertura em todas as declarações condicionais do tipo *if* e *switch*. Essa métrica contabiliza o

[1] O JaCoCo está disponível através do endereço <https://www.eclemma.org/jacoco/>.
[2] O PIT está disponível através do endereço <https://pitest.org/>.

número total de ramificações presentes em um método e determina quantas foram executadas ou perdidas. É importante destacar que o tratamento de exceções não é considerado ramificações nesse contexto.

Já no caso do PIT, a métrica utilizada foi o teste de mutação. Essa técnica consiste em gerar variantes através de pequenas modificações no código a ser testado de modo a simular defeitos no código. Essas variantes são denominadas mutantes. No teste de mutação, o objetivo é avaliar a quantidade de mutantes que são identificados pelo conjunto de testes. A qualidade dos testes é, então, avaliada pela porcentagem de mutações eliminadas em relação ao total, indicando a capacidade do conjunto de casos de teste em identificar defeitos. Utilizamos o PIT para executar o processo do teste de mutação e extrair a métrica do escore de mutação.

Essas métricas foram selecionadas por refletir dois aspectos complementares da qualidade dos testes: abrangência estrutural, nos casos dos testes de cobertura de instruções e ramificações e efetividade na detecção de defeitos, no caso do teste de mutação.

Essa estratégia metodológica busca garantir uma abordagem coerente com os objetivos do trabalho, contribuindo não apenas para o avanço do conhecimento científico, mas também oferecendo subsídios práticos e replicáveis para profissionais e pesquisadores interessados em empregar LLMs na automação e melhoria dos testes de software.

5 DIRETRIZES PARA O USO DE LLMS NO TESTE DE SOFTWARE

Este capítulo apresenta diretrizes e boas práticas para o uso de LLMs em atividades do Teste de Software. Este guia tem como objetivo utilizar LLMs, como o Gemini Pro, por exemplo, para apoiar a geração automatizada de artefatos de teste no processo de desenvolvimento de software. A proposta é criar prompts genéricos e reutilizáveis que orientem a execução de tarefas específicas relacionadas a testes.

A ideia central é aproveitar a capacidade desses modelos em compreender linguagem natural e gerar código ou análises com base em descrições fornecidas pelo usuário. Cada seção do guia apresenta: o nome da tarefa, uma descrição da atividade, um prompt genérico e um exemplo de aplicação prática.

O guia está organizado em três seções iniciais, que abordam diferentes tipos de testes: Testes Funcionais, Testes Estruturais e Testes Automatizados. É importante destacar que as tarefas apresentadas nas seções de testes funcionais e testes estruturais concentram-se no projeto de casos de teste, ou seja, na definição de entrada, condições e resultados esperados. Já as atividades da seção de testes automatizados tem como objetivo a criação de scripts para automatizar a execução dos testes, permitindo sua execução repetida e sistemática.

Para a elaboração da estrutura dos prompts, utilizamos como base os trabalhos previamente apresentados na seção de Trabalhos Relacionados, com destaque para o artigo *LLM-Powered Test Case Generation for Detecting Tricky Bugs,* de Liu K. et al., entre outros que exploram o uso de LLMs na geração de testes. Desse modo, adotamos padrões de engenharia de prompt amplamente reconhecidos, como o uso de persona e template estruturados.

5.1 Teste Funcional

Nesta seção, serão apresentadas tarefas para projetar casos de teste utilizando técnicas de teste funcional, como o particionamento em classes de equivalência e análise do valor limite.

Tarefa 1 : Teste de Particionamento em Classe de Equivalência

O objetivo desta tarefa é utilizar o LLM para projetar casos de teste utilizando a técnica de Particionamento em Classes de Equivalência. Desse modo, o LLM deve gerar casos de teste que explorem diferentes cenários do programa, como testes com dados válidos e inválidos. Isso permite testar o sistema de forma eficiente e repetível, reduzindo o esforço

manual e garantindo maior cobertura e confiabilidade na verificação de funcionalidades e regras do software.

Quadro 2 - Teste de particionamento em classe de equivalência

Atue como um Testador de Software.
Gere casos de teste para o programa [nome do programa X] com base na documentação/especificação fornecida que deve está anexada e nos critérios de particionamento de classes de equivalência.
Certifique-se de incluir casos com entradas válidas e inválidas, apresentando cada classe identificada.
Para cada caso, forneça: ID do teste, objetivo, entrada, saída esperada, ambiente de teste.

Fonte: Autoria própria (2025)

Tarefa 2 : Teste de Análise de Valor Limite

O objetivo desta tarefa é utilizar o LLM para projetar casos de teste utilizando a técnica de Análise do Valor Limite. Assim, os casos de teste gerados devem explorar valores nos extremos e adjacentes aos intervalos válidos de cada parâmetro de entrada. O objetivo é identificar possíveis falhas que ocorram nas bordas do domínio de entrada, onde erros são estatisticamente mais prováveis. Essa abordagem contribui para uma validação mais eficaz e direcionada.

Quadro 3 - Teste de análise de valor limite

Atue como um Testador de Software.
Gere casos de teste para o programa [nome do programa X] com base na documentação/especificação fornecida que está anexada e nos critérios de análise de valor limite.
Certifique-se de incluir valores imediatamente abaixo, no limite e acima dos limites permitidos para cada parâmetro.
Para cada caso, forneça: ID do teste, objetivo, entrada, saída esperada, ambiente.

Fonte: Autoria própria (2025)

5.2 Teste Estrutural

Nesta seção, serão apresentadas tarefas para projetar casos de teste utilizando técnicas de teste estrutural, como a cobertura de linhas, cobertura de ramificações e cobertura de condições.

Tarefa 3 : Teste de Cobertura de Linha

O objetivo desta tarefa é utilizar o LLM para projetar casos de teste utilizando a técnica de cobertura de linhas. Dessa forma, devem ser gerados casos de testes que percorrem cada instrução do programa. O objetivo é identificar linhas não executadas que podem ocultar defeitos.

Quadro 4 - Teste de cobertura de linha

Atue como um Testador de Software.
Gere casos de teste para o programa [nome do programa X] com base nos critérios teste de cobertura de linha.
[Aqui você vai inserir o trecho do código-fonte].
Certifique-se de que os testes gerados executem todas as linhas do código-fonte ao menos uma vez.
Garanta que nenhuma linha relevante do programa fique sem execução durante os testes.
Identifique linhas não cobertas e crie casos para validar seu comportamento.
Minimize riscos de falhas assegurando uma cobertura completa de linhas.
Para cada caso, forneça: ID do teste, objetivo, entrada, saída esperada, ambiente.

Fonte: Autoria própria (2025)

Tarefa 4 : Teste de Cobertura de Ramificação.

O objetivo desta tarefa é utilizar o LLM para projetar casos de teste utilizando a técnica de cobertura de ramificações para gerar casos de testes que validam tanto os caminhos verdadeiros quantos os falsos de cada estrutura de controle. O objetivo é identificar comportamentos incorretos ou não testados em diferentes fluxos lógicos.

Quadro 5 - Teste de cobertura de ramificação

Atue como um Testador de Software.
Gere casos de teste para o programa [nome do programa X] com base nos critérios teste de cobertura de ramificação.
[Aqui você vai inserir o trecho do código-fonte].
Certifique-se de que os testes gerados exercitem todas as decisões condicionais, cobrindo tanto os caminhos verdadeiros quanto os falsos.
Garanta que todas as ramificações do código sejam executadas ao menos uma vez durante os testes.
Identifique decisões não cobertas e crie casos para validar seus resultados.

Minimize riscos de falhas assegurando uma cobertura completa de linhas.
Para cada caso, forneça: ID do teste, objetivo, entrada, saída esperada, ambiente.

Fonte: Autoria própria (2025)

Tarefa 5 : Teste de Cobertura de Condições.

O objetivo desta tarefa é utilizar o LLM para projetar casos de teste utilizando a técnica de Cobertura de Condições. Dessa forma, o objetivo é gerar testes que garantem que cada condição booleana dentro das expressões condicional seja avaliada ao menos uma vez como verdadeira e uma vez como falsa. Essa abordagem permite identificar falhas sutis em decisões complexas, promovendo uma análise mais detalhada do comportamento lógico do programa e aumentando a confiabilidade dos testes.

Quadro 6 - Teste de cobertura de condições

Atue como um Testador de Software.
Gere casos de teste para o programa [nome do programa X] com base nos critérios teste de cobertura de condições.
[Aqui você vai inserir o trecho do código-fonte]
Considere todas as subcondições booleanas presentes nas expressões condicionais compostas.
Para cada subcondição, crie pelo menos dois casos de teste: um em que ela seja avaliada como verdadeiro e outro em que seja avaliada com falsa.
Para cada caso, forneça: ID do teste, objetivo, entrada, saída esperada, ambiente.

Fonte: Autoria própria (2025)

5.3 Teste Automatizados

Nesta seção, serão apresentadas tarefas para gerar scripts de teste automatizados.

Tarefa 6 : Gerar Script de Teste Automatizado.

O objetivo desta tarefa é utilizar o LLM para gerar scripts de teste automatizados para um programa cujo código-fonte é passado como entrada. Nesta tarefa não é especificada nenhuma técnica para projetar os casos de teste. Desse modo, o LLM se encarrega não apenas de gerar o script de teste automatizado, mas também de projetar os casos de teste que vão compor o conjunto de testes. O prompt orienta a criação de scripts válidos, claros e

executáveis, utilizando frameworks de teste adequado e assertivos que validem o comportamento real da implementação.

Quadro 7 - Gerar script de teste automatizado

Atue como um Testador de Software.
Gerar Script de Teste Automatizado para o programa [nome do programa X] com base no trecho do código-fonte anexado a seguir [código-fonte]
Utilize o framework [nome do framework] da linguagem [nome da linguagem]
O script gerado deve ser válido, executável e claro, com assertivas precisas, cobertura de diferentes cenários e compatível com o framework.
Certifique-se de que o teste seja reprodutível.
Para cada caso, forneça: ID do teste, objetivo, entrada, saída esperada, ambiente de teste.

Fonte: Autoria própria (2025)

Tarefa 7: Gerar Script de Teste Automatizado a partir de uma especificação de casos de casos de teste

O objetivo desta tarefa é utilizar o LLM para automatizar casos de teste que tenham sido previamente projetados. Desse modo, deve ser passada a especificação de casos de teste e o programa a ser testado como entrada para o LLM para que ele gere um script de teste automatizado com base na especificação. Assim, busca-se automatizar a produção de testes confiáveis, aumentando a eficiência, reduzindo o esforço manual e elevando significativamente a qualidade do processo de validação.

Quadro 8 - Gerar script de teste automatizado a partir de uma especificação

Atue como um Testador de Software.
Gerar Script de Teste Automatizado para o programa [nome do programa X] , cujo a especificação de casos de teste e código-fonte se encontram [os artefatos podem ser anexados ou colocados abaixo].
Utilize o framework [nome do framework] da linguagem [nome da linguagem]
O script deve ser válido, executável e claro, com assertivas precisas e compatível com o framework especificado.
Certifique-se de que o teste seja reprodutível.
Para cada caso, forneça: ID do teste, objetivo, entrada, saída esperada, ambiente de teste.

Fonte: Autoria própria (2025)

Ao longo deste capítulo, exploramos como as LLM podem ser empregadas para auxiliar as atividades de Teste de Software. Descrevemos um conjunto de tarefas organizadas em Testes Funcionais, Testes Estruturais e Testes Automatizados. Tendo em vista a fundamentação teórica e as diretrizes apresentadas, o próximo passo importante é a avaliação prática deste. Portanto, o capítulo seguinte será dedicado à descrição do experimento que avalia a qualidade e a utilidade dos artefatos de testes gerados com o suporte dos LLMs.

6 EXPERIMENTO DE AVALIAÇÃO

Este capítulo apresenta os resultados de um experimento realizado para avaliar a qualidade e a eficiência das diretrizes para o uso de LLMs em atividades do Teste de Software propostas no capítulo anterior. As seções a seguir apresentam o processo de execução do experimento, resultados dos experimentos e uma análise dos resultados.

6.1 Processo de Execução dos Experimentos

A etapa experimental deste trabalho foi conduzida de forma estruturada, compreendendo desde a seleção dos programas utilizados até a análise detalhada das métricas obtidas. Inicialmente, foram escolhidos quatro programas amplamente utilizados na literatura de Teste de Software: CountWords(CW), PlanningPoker(PP), SubstringsBetween(SB) e TriangleType(TT). Esses programas foram selecionados por sua simplicidade e por possibilitarem a aplicação prática de critérios funcionais e estruturais, conforme indicam autores como Delamaro, Maldonado e Jino (2007) e Aniche (2022). Essa escolha visou garantir uma base consistente para avaliar a capacidade das LLMs em gerar artefatos de teste, além de permitir a replicação dos experimentos em trabalhos futuros.

Outros aspectos importantes referem-se aos critérios de testes, que representam as estratégias utilizadas para a geração dos casos de testes. Dentro os critérios adotados, destacam-se: particionamento em classe de equivalência (PCE), análise de valor limite (AVL), cobertura de linhas (CL), cobertura de ramificações (CR), cobertura de condições (CC) e, adicionalmente testes realizados sem critérios (SC) definidos. Cada um desses critérios visa garantir diferentes perspectivas de verificação e níveis de abrangência no teste do sistema. Para cada critério, foram contabilizados os casos de teste gerados (CT), os inválidos (CTI) e os falhos (CTF), que indicam possíveis defeitos no código. Também foi avaliado o nível de automação (NA), classificado em total, parcial ou inexistente, conforme a necessidade de ajustes ou falhas na execução. Casos inválidos ou falhos foram mantidos apenas como comentários nos scripts.

Em seguida, foi realizada a aplicação das tarefas definidas no guia elaborado durante a pesquisa. Para cada programa, executado de forma isolada as tarefas 1 a 5, voltadas ao projeto de casos de teste segundo critérios funcionais (como Particionamento em Classes de Equivalência e Análise de Valor Limite) e estruturais (como Cobertura de Linha, Cobertura de Ramificações e Cobertura de Condições). Após a geração dos conjuntos de testes para cada critério, proceder à execução da tarefa 7, com o objetivo de converter esses casos em scripts

automatizados, utilizando um framework compatível com a linguagem dos programas avaliados. Esse processo foi realizado repetidamente para cada conjunto gerado, o que resultou na produção de um volume considerável de artefatos, evidenciando o esforço envolvido. Além disso, realizei a tarefa 6, que consistiu em gerar scripts de teste sem a especificação de um critério, buscando avaliar a capacidade do LLM em propor soluções livres e comparar esses resultados com os obtidos a partir de instruções mais específicas.

A fase seguinte correspondeu à execução dos scripts de teste gerados. Para essa etapa, foi necessário efetuar ajustes manuais em alguns scripts, a fim de corrigir inconsistências e garantir sua execução adequada no ambiente de testes. Em seguida, as métricas foram coletadas por meio das ferramentas JaCoCo e PIT, amplamente utilizadas em estudos de qualidade de software. A ferramenta JaCoCo forneceu as métricas de cobertura de instruções e cobertura de ramificações, enquanto a ferramenta PIT permitiu a obtenção da pontuação de mutação, utilizada para mensurar a efetividade das suítes na detecção de falhas. A escolha dessas métricas se deu pelo fato de refletirem, de forma complementar, a abrangência e a qualidade dos testes gerados.

Por fim, os dados coletados foram analisados com o objetivo de identificar padrões, limitações e pontos de melhoria. Essa análise incluiu a consolidação dos resultados em tabelas comparativos, além da discussão das métricas obtidas em cada critério de teste aplicado. Esse processo possibilitou não apenas mensurar a eficácia das LLMs na geração de testes automatizados, mas também compreender os desafios encontrados, como a ocorrência de casos inválidos e a necessidade de ajustes manuais nos scripts.

6.2 Resultados dos Experimentos

Os resultados dos experimentos estão sumarizados na Tabela 1 apresentada logo abaixo.

Quadro 9: Métricas de avaliação dos testes gerados

Programas	Critérios	CT	CTI	CTF	NA	Instrução	Ramificação	Mutação
CW	PCE	16	3	1	Parcial	100%	100%	100%
	AVL	16	10	1	Parcial	100%	91%	100%
	CL	7	2	0	Parcial	100%	83%	100%
	CR	13	4	0	Parcial	97%	83%	90%
	CC	18	0	0	Total	100%	100%	100%
	SC	15	1	1	Parcial	100%	100%	100%
PP	PCE	9	0	0	Total	100%	100%	100%
	AVL	10	0	0	Total	100%	100%	100%
	CL	8	0	0	Total	100%	100%	100%
	CR	10	0	0	Total	100%	100%	100%
	CC	15	0	0	Total	100%	100%	100%
	SC	11	0	0	Total	100%	100%	100%
SB	PCE	17	1	1	Parcial	100%	100%	85%
	AVL	16	0	3	Parcial	100%	100%	85%
	CL	17	0	0	Total	100%	100%	85%
	CR	17	2	0	Parcial	100%	100%	85%
	CC	19	1	0	Parcial	100%	100%	80%
	SC	20	3	0	Parcial	100%	100%	85%
TT	PCE	21	0	0	Total	93%	100%	88%
	AVL	20	0	1	Parcial	93%	100%	88%
	CL	11	0	0	Total	93%	100%	76%
	CR	18	0	0	Total	93%	100%	80%
	CC	28	0	0	Total	93%	100%	80%
	SC	20	0	0	Total	93%	95%	80%

Fonte: Autoria própria(2025)

Nas subseções a seguir serão detalhados os resultados obtidos para cada programa testado, levando em conta os casos de teste gerados por cada critério e os scripts de teste automatizados. O objetivo é descrever de forma sucinta como cada conjunto de testes foi construído, automatizado e executado, destacando o comportamento observado durante os experimentos. Cada subseção detalha os resultados obtidos em um programa, onde serão detalhados os critérios utilizados: particionamento de classes de equivalência, análise de valor limite, cobertura de linhas, cobertura de ramificações, cobertura de condições e testes sem critérios definidos. Em cada caso, são identificadas as possíveis adaptações necessárias para execução, a abordagem adotada para a geração dos scripts e automação dos casos de testes e os resultados obtidos, incluindo casos inválidos e falhas encontradas.

Esse detalhamento permite avaliar a eficiência de cada critério na geração de casos de testes e na detecção de comportamento inesperado, bem como compreender o impacto do nível de automação na execução dos testes.

6.2.1 CountWords

O programa CountWords tem como objetivo contar a quantidade de palavras que terminam com as palavras ‘s’ e/ou ‘r’ em uma string. Para isso, percorrer cada caractere da entrada, verificando se o caractere anterior é uma letra e se atende ao critério definido, incrementando um contador quando a condição é satisfeita. Esse exemplo é comumente utilizado para demonstrar a aplicação de testes estruturais, especialmente cobertura de código, abordando critérios como linha, ramificações e condições. A implementação está disponível no livro do autor Aniche (2022).

6.2.1.1 Particionamento de Equivalência de Classes

Foi necessário apenas instanciar o objeto da classe CountWords. A criação dos casos de teste foi baseada na Tarefa 1, a partir da qual foram elaborados os casos correspondentes. Para a automação do script, utilizou-se a Tarefa 7, que implementou a execução dos mesmos 16 casos e acrescentou um caso adicional que não estava previsto na especificação. Este caso extra foi comentado no código para não interferir nos resultados gerados pelo LLM.

Os testes automatizados foram executados, porém nem todos obtiveram sucesso: três foram considerados inválidos por não seguirem a especificação, e um apresentou falha mesmo estando em conformidade com a especificação, devido ao fato de o código não atender ao comportamento esperado.

6.2.1.2 Análise de Valor Limite

Não foi necessário realizar alterações para a execução dos casos de teste. A criação dos casos teve como base a Tarefa 2, a partir da qual foram gerados os testes correspondentes. A automação foi realizada por meio da Tarefa 7, que implementou a execução dos mesmos 16 casos.

Após a execução, constatou-se que nem todos os testes obtiveram sucesso: 10 foram considerados inválidos por não seguirem a especificação e 1 apresentou falha, mesmo atendendo à especificação, devido à implementação não corresponder ao comportamento esperado

6.2.1.3 Cobertura de Linha

Não foi necessário realizar alterações para a execução dos casos de teste. A criação dos casos foi baseada na Tarefa 3, e a automação foi realizada pela Tarefa 7, contemplando os mesmos 7 casos.

Os testes foram executados, porém 2 foram considerados inválidos por não seguirem a especificação.

6.2.1.4 Cobertura de Ramificações

Não foi necessário realizar alterações para a execução dos casos de teste. A criação dos casos teve como referência a Tarefa 4, sendo todos automatizados por meio da Tarefa 7, totalizando 13 casos.

Durante a execução, verificou-se que 4 testes foram considerados inválidos por não seguirem a especificação.

6.2.1.5 Cobertura de Condições

Não foi necessário realizar alterações para a execução dos casos de teste. A criação dos casos foi baseada na Tarefa 5, e a automação foi realizada pela Tarefa 7, abrangendo 18 casos.

Todos os testes foram executados com sucesso, sem falhas ou invalidações.

6.2.1.6 Sem Critérios

Não houve necessidade de ajustes para a execução dos casos de teste. A automatização foi realizada na Tarefa 6, etapa em que não foram fornecidos casos de teste prontos, sendo necessário criar os testes, totalizando 15 casos.

Durante a execução, verificou-se que 1 caso foi considerado inválido por não atender à especificação e outro apresentou falha, mesmo estando de acordo com a especificação, o que indica que a implementação não cumpre o comportamento esperado.

6.2.2 PlanningPoker

O programa PlanningPoker implementou uma prática comum em metodologias ágeis, que consiste em uma dinâmica com a equipe de desenvolvimento para estimar o esforço necessário para implementar uma funcionalidade no software. Desse modo, esse programa recebe como entrada uma lista de estimativas de desenvolvedores e deve identificar as estimativas mais extremas (menos e maior) para que a equipe possa discutir as diferenças. A lógica consiste em percorrer a lista de estimativas, armazenando os valores mínimo e máximo, e retornando os respectivos nomes dos desenvolvedores. Esse exemplo é comumente utilizado para demonstrar a aplicação de testes estruturais, especificamente cobertura de código, abordando critérios como linha, ramificação e condições. A implementação está disponível no livro do autor Aniche (2022).

6.2.2.1 Particionamento de Equivalência de Classes

Foi necessário apenas instanciar o objeto da classe PlanningPoker. Para a criação de casos de testes, utilizou-se a tarefa 1, a partir da qual foram gerados casos de testes. Para a automação do script, utilizou-se a tarefa 7, que automatizou os mesmos 10 casos e 1 caso adicional que não havia sido especificado.

Todos os casos de testes foram executados com sucesso, sem detecção de falhas ou testes inválidos.

6.2.2.2 Análise de Valor Limite

Não foi necessário fazer nenhuma alteração, para executar os casos de testes. Para a criação de casos de testes, utilizou-se a tarefa 2, a partir da qual foram gerados casos de testes. Para a automação do script, utilizou-se a tarefa 7, que automatizou os mesmos 9 casos e 1 caso adicional que não havia sido especificado na especificação.

Todos os casos de testes foram executados com sucesso, sem detecção de falhas ou testes inválidos.

6.2.2.3 Cobertura de Linha

Não foi necessário fazer nenhuma alteração, para executar os casos de testes. Para a criação de casos de testes, utilizou-se a tarefa 3, a partir da qual foram gerados casos de testes. Para a automação do script, utilizou-se a tarefa 7.

Todos os casos de testes foram executados com sucesso, sem detecção de falhas ou testes inválidos.

6.2.2.4 Cobertura de Ramificações

Não foi necessário fazer nenhuma alteração, para executar os casos de testes. Para a criação de casos de testes, utilizou-se a tarefa 4, a partir da qual foram gerados casos de testes. Para a automação do script, utilizou-se a tarefa 7, que automatizou os mesmos 10.

Todos os casos de testes foram executados com sucesso, sem detecção de falhas ou testes inválidos.

6.2.2.5 Cobertura de Condições

Não foi necessário fazer nenhuma alteração, para executar os casos de testes. Para a criação de casos de testes, utilizou-se a tarefa 5, a partir da qual foram gerados casos de testes. Para a automação do script, utilizou-se a tarefa 7, que automatizou os mesmos 15.

Todos os casos de testes foram executados com sucesso, sem detecção de falhas ou testes inválidos.

6.2.2.6 Sem Critérios

Não houve necessidade de ajustes para a execução dos casos de teste. A automatização foi realizada na Tarefa 6, etapa em que não foram fornecidas especificações de casos de testes, sendo necessário criar os testes, totalizando 10 casos.

Todos os casos de testes foram executados com sucesso, sem detecção de falhas ou testes inválidos.

6.2.3 SubstringsBetween

O método substring Between busca extrair todas as substrings de uma string principal que estejam delimitadas por duas tags (strings de abertura e fechamento) fornecidas pelo usuários. Retorna um array com as ocorrências entradas ou null quando não há correspondências. Esse exemplo é comumente utilizado para aplicar testes baseados em especificação, incluindo particionamento em classes de equivalência e análise de valores limites, bem como para explorar combinações e casos inválidos. A implementação está disponível no livro do autor Aniche (2022).

6.2.3.1 Particionamento de Equivalência de Classes

Foi necessário apenas instanciar os objetos da classe StringUtils e declarado uma variável. A criação dos casos de testes foram baseados na tarefa 1, a partir da qual foram elaborados os casos correspondentes. A automação foi realizada por meio da tarefa 7, que implementou a execução dos mesmos 17 casos.

Os testes automatizados foram executados, porém nem todos obtiveram sucesso: um foi considerado invalido por não seguir a especificação, e um apresentou falha, mesmo atendendo a especificação, devido ao fato do código não implementar o comportamento esperado.

6.2.3.2 Análise de Valor Limite

Não foi necessário fazer nenhuma alteração para a execução dos casos de testes. A criação dos casos de testes foram baseadas na tarefa 2, a partir da qual foram gerados os casos correspondem, A automação foi realizada por meio da tarefa 7, que implementou a execução dos mesmo 16 casos.

Após a execução, constatou-se que nem todos os testes obtiveram sucesso: três apresentaram falhas, mesmo atendendo a especificação, devido a implementação não correspondendo ao comportamento esperado.

6.2.3.3 Cobertura de Linha

Não foi necessário fazer nenhuma alteração para a execução dos casos de testes. A criação foi baseada na tarefa 3, e a automação foi realizada por meio da tarefa 7, abrangendo 17 casos.

Todos os casos de testes foram executados com sucesso, sem detecção de falhas ou testes inválidos.

6.2.3.4 Cobertura de Ramificações

Não foi necessário fazer nenhuma alteração para a execução dos casos de testes. A criação foi baseada na tarefa 4, e a automação foi realizada por meio da tarefa 7, abrangendo 17 casos.

Os testes automatizados foram executados, porém nem todos obtiveram sucesso: dois foram considerados inválidos por não seguirem a especificação.

6.2.3.5 Cobertura de Condições

Não foi necessário fazer nenhuma alteração para a execução dos casos de testes. A criação dos casos de testes foi baseada na tarefa 5, a partir da qual foram elaborados os casos correspondentes. A automação foi realizada por meio da tarefa 7, que implementou a execução dos mesmo 19 casos.

Os testes automatizados foram executados, porém nem todos obtiveram sucesso: um foi considerado invalido por não seguir a especificação.

6.2.3.6 Sem Critérios

Não houve necessidade de ajustes para a execução dos casos de teste. A automatização foi realizada na Tarefa 6, etapa em que não foram fornecidos scripts prontos, sendo necessário criar os testes, totalizando 20 casos.

Todos os testes foram executados com sucesso, sem falhas ou invalidações.

6.2.4 TriangleType

O programa Triangle Type contém o método triang, responsável por classificar um triângulo com base nos comprimentos de seus três lados. A função determina se o triângulo é equilátero, isósceles, escaleno ou invalido (quando não satisfaz a propriedade triangular). Esse exemplo é comumente utilizado para ilustrar a técnica de particionamento de classe de equivalência e análise de valores limites, destacando a importância do particionamento do domínio de entrada. A implementação e explicações estão no livro dos autores Ammann e Jeff Offutt (2017).

6.2.4.1 Particionamento de Equivalência de Classes

Não foi necessário realizar alterações para a execução dos casos de teste. A criação dos casos foi baseada na Tarefa 1, e a automação foi realizada pela Tarefa 7, abrangendo 21 casos.

Todos os testes foram executados com sucesso, sem falhas ou invalidações.

6.2.4.2 Análise de Valor Limite

Não foi necessário realizar alterações para a execução dos casos de teste. A criação dos casos foi baseada na Tarefa 2, e a automação foi realizada pela Tarefa 7, contemplando os mesmos 7 casos.

Os testes foram executados, porém 1 teste falhou, mesmo atendendo à especificação, indicando que a implementação não cumpre o comportamento esperado.

6.2.4.3 Cobertura de Linha

Não foi necessário realizar alterações para a execução dos casos de teste. A criação dos casos foi baseada na Tarefa 3, e a automação foi realizada pela Tarefa 7, abrangendo 11 casos.

Todos os testes foram executados com sucesso, sem falhas ou invalidações.

6.2.4.4 Cobertura de Ramificações

Não foi necessário realizar alterações para a execução dos casos de teste. A criação dos casos foi baseada na Tarefa 4, e a automação foi realizada pela Tarefa 7, abrangendo 18 casos.

Todos os testes foram executados com sucesso, sem falhas ou invalidações.

6.2.4.5 Cobertura de Condições

Não foi necessário realizar alterações para a execução dos casos de teste. A criação dos casos foi baseada na Tarefa 5, e a automação foi realizada pela Tarefa 7, abrangendo 28 casos.

Todos os testes foram executados com sucesso, sem falhas ou invalidações.

6.2.4.6 Sem Critérios

Não houve necessidade de ajustes para a execução dos casos de teste. A automatização foi realizada na Tarefa 6, etapa em que não foram fornecidos scripts prontos, sendo necessário criar os testes, totalizando 20 casos.

Todos os testes foram executados com sucesso, sem falhas ou invalidações.

6.3 Análise dos Resultados

Para avaliar a efetividade do uso de LLMs na geração e execução de casos de testes, foram realizados experimentos com quatro programas distintos: CountWords, PlanningPoker, SubstringsBetween e TriangleType. Esses programas foram selecionados por apresentarem características variadas, abrangendo desde métodos simples de manipulação de strings até funções com complexas com múltiplos caminhos condicionais, permitindo testar diferentes níveis de cobertura.

O objetivo desta análise é compreender a capacidade das LLMs de criar casos de testes aderentes às especificações, atingir níveis elevados de cobertura estrutural e mutação, além de viabilidade de automação dos scripts gerados. Para isso, foram analisadas métricas como número de casos gerados, ocorrências de erros (casos inválidos e falhas), cobertura de instruções, cobertura de ramificações, análise de mutação e nível de automação alcançado.

Inicialmente, optou-se por utilizar códigos relativamente pequenos para os experimentos, com o intuito de avaliar como a LLM se comportaria em estruturas mais simples antes de avançar para cenários mais complexos. Essa abordagem permite identificar o nível de precisão e qualidade dos casos como etapa preliminar para aplicações em sistemas de maior porte.

Embora a análise anterior tenha abordado os critérios tradicionais aplicados pela LLM, é importante observar o comportamento dos testes sem aplicação de critério específico (SC). Essa categoria é relevante porque demonstra como o modelo se comporta quando não recebe diretrizes para geração, permitindo avaliar se a simples instrução de “gerar casos de teste” é suficiente para atingir níveis aceitáveis de qualidade.

A quadro 10 apresenta os resultados obtidos para SC nos quatro programas avaliados, considerando número total de casos de teste (CT), casos inválidos (CTI), casos falhos (CTF), coberturas de instrução, ramificações e mutações.

Quadro 10: Resultados para casos sem critérios

Programa	CT	CTI	CTF	Instruções	Ramificação	Mutação
CountWords	15	1	1	100%	100%	100%
PlanningPoker	11	0	0	100%	100%	100%
SubstringsBetween	20	3	0	100%	100%	85%
TriangleType	20	0	0	93%	95%	80%

Fonte: Autoria própria(2025)

A análise revela que, mesmo sem diretrizes explícitas, os testes gerados pela LLM atingiram cobertura estrutural elevada para instruções e ramificações (entre 93% e 100%), desempenho semelhante ao observado com critérios específicos. Em alguns casos, como CountWords e PlanningPoker, os resultados foram praticamente perfeitos, com 100% em todas as métricas, incluindo mutação, indicando que comandos genéricos podem gerar testes altamente eficazes em cenários simples.

No entanto, a ausência de critério não garante qualidade consistente. Programas mais complexos, como SubstringsBetween e TriangleType, apresentaram quedas significativas na cobertura de mutação (85% e 80%, respectivamente), sugerindo menor capacidade de detecção de defeitos sem a aplicação de estratégias específicas. Isso é um ponto crítico, pois a mutação é um indicador mais robusto da efetividade do teste do que a mera cobertura estrutural.

Outro aspecto relevante é o aumento no número de testes gerados em alguns casos, 20 para SubstringsBetween e TriangleType, sem ganho proporcional em cobertura estrutural ou mutação, o que implica redundância e execução mais lenta. Além disso, a ocorrência de casos inválidos, 1 em CountWords e 3 em SubstringsBetween, indica que a ausência de critério não elimina inconsistências, exigindo ajustes manuais que reduzem a automação plena.

Antes de analisar individualmente cada programa, é fundamental observar a performance das LLMs sob a ótica dos critérios aplicados. Está visão consolidada permite identificar quais estratégias foram mais eficientes e consistentes na geração de casos, quais apresentam maior índice de erros e como as métricas de cobertura e mutação se comportaram de forma geral. Essa análise fornece um panorama global sobre a efetividade do LLM na aplicação de técnicas tradicionais de testes. A Tabela 3 apresenta a média dos resultados agregados por critério de cobertura.

Quadro 11: Médias por critério

Critérios	CT	CTI	CTF	Instruções	Ramificação	Mutação
PCE	15,75	1	0,5	98,25%	100%	93,25%
AVL	15,5	2,75	1	98,25%	97,75%	93,25%
CL	10,75	0,5	0	98,25%	95,75%	90,25%
CR	14,5	1,5	0	97,5%	95,75%	88,75%
CC	20	0	0,25	98,25%	100%	90%
SC	16,25	0,25	0,25	98,25%	98,75%	91,25%

Fonte: Autoria própria(2025)

Os dados revelam que as LLMs conseguiram aplicar corretamente os critérios propostos, gerando uma quantidade expressiva de casos para todos os cenários. O destaque positivo vai para cobertura de linha (CL), que produziu menos casos de teste (10,25 em média) com quase zero erros, o que se justifica por ser o critério mais simples, exigindo apenas a execução de todas as linhas do código. Por outro lado, a análise de valor limite (AVL) apresentou maior média de inválidos (2,75 em média), sinalizando dificuldade na interpretação de valores externos. Ainda assim, a cobertura manteve-se elevada: 97,75% para instrução e 97,75% para ramificação, indicando eficiência estrutural.

A análise de mutação, variando entre 88,75% a 93,25%, reforça a qualidade dos casos, pois testes que conseguem matar mutantes indicam forte poder de detecção de falhas. Esses resultados demonstram que as LLMs não apenas geram casos de testes sistemáticos, mas também os estruturam com critérios que garantem alta cobertura. Outro ponto a ser destacado é a cobertura de condições (CC). Embora tenha gerado um número significativamente maior de casos de testes, esse volume não resultou em coberturas superiores em relação aos demais critérios, o que é o que realmente importa. Isso indica que o maior esforço, na geração de testes, não necessariamente se traduz em melhores resultados, podendo ser visto como uma desvantagem em comparação com os outros critérios, como é o caso desse critério.

Após observar os resultados gerais e as médias por critérios, é necessário compreender o impacto dos casos de testes inválidos nos programas analisados. Esses casos representam situações em que os testes gerados pela LLM não puderam ser executados devido a inconsistências ou divergências com a especificação. Para evitar falhas de execução, esses testes foram comentados no código, permitindo que a automação prosseguisse.

Contudo, essa prática traz implicações negativas importantes: reduz a confiabilidade do conjunto de testes, pois parte da lógica prevista não foi validada; pode mascarar métricas de cobertura, criando falsa sensação de completude; e exige intervenção manual, diminuindo o ganho esperado com a automação. Os resultados dos testes inválidos são sumarizados na Tabela 4 apresentada abaixo.

Quadro 12: Impacto dos casos inválidos por programas

Programa	% inválidos	Casos Comentados	Impacto
CountWords	23,5%	20	Alto, compromete a automação
PlanningPoker	0%	0	Nenhum impacto
SubstringsBetween	6,60%	7	Moderado: ajustes manuais

TriangleType	0%	0	Nenhum impacto

Fonte: Autoria própria(2025)

A análise revela que CountWords apresentou o cenário mais crítico, com 23,5% de casos inválidos, todos comentados para permitir a execução. Essa intervenção não apenas comprometeu a automação, como também reduziu a efetividade dos testes, pois uma parcela significativa dos casos não foram executados. Além disso, a prática de comentar casos pode levar a interpretações equivocadas dos resultados, já que métricas de cobertura podem não refletir a real capacidade de detecção de falhas.

O SubstringsBetween apresentou impacto moderado, com 6,60% de casos comentados, exigindo ajustes manuais que também reduzem os benefícios da automação. Em contraste, PlanningPoker e TriangleType representam o cenário ideal com zero casos inválidos, execução completa e automação integral.

Esses dados indicam que, embora as LLMs tenham potencial para automatizar testes, a confiabilidade é fortemente afetada quando há inconsistência nos casos gerados, tornando indispensável a revisão humana. A presença de casos inválidos compromete diretamente a eficiência do processo e pode induzir equipes a uma falsa percepção de qualidade.

Diante dos casos inválidos, outro fator que influencia diretamente a eficiência dos testes gerados pelas LLMs são os casos de testes que falharam (CTF). Esses casos foram executados, mas resultaram em falhas, o que indica, na maioria das vezes, a presença de defeitos reais no código-fonte. Independente do caso, a presença de falhas impactam a credibilidade do processo e pode exigir esforço adicional para análise e depuração.

Quadro 13: Impacto das falhos por programas

Programa	% falhas	Casos Comentados	Impacto
CountWords	3,53%	3	Moderado: precisa de análise manual
PlanningPoker	0%	0	Nenhum impacto
SubstringsBetween	3,77%	4	Ato: indica risco de inconsistência
TriangleType	0,94%	1	Baixo impacto

Fonte: Autoria própria(2025)

A análise das falhas revela que, mesmo em um contexto de alta cobertura estrutural ocorreram situações que comprometem a confiabilidade do processo. O SubstringsBetween apresentou a maior taxa de falhas (3,77%), com 4 casos que não produziram os resultados esperados. Esse cenário sugere duas hipóteses: ou o código original possui comportamentos

divergentes da especificação, ou as instruções geradas pelo LLM continham inconsistência. Em ambos os casos, a automação perde parte de sua eficiência, pois exige investigação manual para determinar a causa raiz.

O CountWords, com três falhas (3,53%), também evidencia um impacto relevante, especialmente considerando que esse programa já tinha altos índices de casos inválidos. Isso reforça a ideia de que códigos mais complexos ou com maior dependência de condições internas tendem a gerar mais problemas quando testados automaticamente por LLM.

Em contraste, o PlanningPoker obteve execução ideal, sem falhas ou ajustes, confirmando que cenários mais previsíveis e com lógica simples favorecem a automação completa e confiável. O TriangleType, embora tenha apresentado uma falha, manteve impacto baixo em função do alto volume de casos válidos e corretos.

A combinação desses dois fatores indica que, embora as LLMs tenham potencial para automatizar testes e gerar alta cobertura, a confiabilidade ainda é limitada por erros de geração e falhas inesperadas. Esses problemas não invalidam a abordagem, mas reforçam a necessidade de supervisão humana para garantir qualidade e segurança no processo.

Após avaliar os critérios de forma global, é essencial analisar como cada programa se comportou individualmente. Essa etapa permite entender se as variações observadas decorrem da complexidade do código, do tipo de lógica implementada ou da robustez do critério aplicado. Essa comparação é relevante porque evidencia os pontos fortes e fracos da abordagem do LLM em diferentes contextos.

Quadro 14: Consolidação por programa

Programa	Total de CT	Total de CTI	Total de CTF	Automação
CountWords	85	20	3	Parcial
PlanningPoker	63	0	0	Total
SubstringsBetween	106	7	4	Parcial
TriangleType	118	0	1	Total

Fonte: Autoria própria(2025)

Os resultados por programa mostraram diferenças significativas. O TriangleType obteve maior número de casos de testes (118), com apenas uma falha, o que confirma a capacidade do modelo lidar com cenários bem estruturados e de maior complexidade, em contrapartida temos o PlanningPoker que obteve menos casos de testes e teve a execução mais desejada, sem ocorrência de erros. Esses dois programas representam extremos opostos em termos de volume de testes, porém ambos demonstram cenários sólidos e consistentes,

evidenciando que a qualidade não está necessariamente vinculada à quantidade de casos gerados, mas sim a aderência das estratégias de teste ao comportamento esperado do código.

Essa análise reforça que a automação total é viável em contextos menos complexos, enquanto códigos com lógicas mais complexas ainda exigem ajustes manuais, embora os benefícios do uso das LLMs sejam evidentes

Com os resultados detalhados por critério e programa, o próximo passo é avaliar a qualidade dos casos gerados sob três perspectivas: cobertura estrutural, capacidade de detecção de falhas (mutação) e conformidade com a especificação. Essa análise é fundamental para validar se os casos não apenas cobrem linhas de código, mas também são eficazes e confiáveis.

Os casos gerados atingiram níveis elevados de cobertura estrutural, superando 95,75% em instruções e ramificações para todos os critérios, garantindo que os caminhos relevantes do código foram exercidos. A cobertura de mutação, com média de 88,75%, confirma que os testes foram eficientes para detectar defeitos, embora ainda exista espaço para aprimorar a força dos casos.

Por outro lado, a conformidade com a especificação mostrou inconsistências, especialmente nos critérios AVL e PCE, onde foram registrados mais casos inválidos. Isso sugere que as LLMs ainda enfrentam desafios para interpretar valores limites e regras especificadas.

Quando a automação, metade dos programas obteve execução completa sem ajustes, enquanto os demais exigiram correções mínimas. Esses resultados indicam que as LLMs são capazes de reduzir significativamente o esforço humano no processo de teste, embora não eliminem a necessidade de validação em cenários complexos.

A análise dos resultados evidencia que as LLMs apresentam um papel relevante no apoio à atividade de teste de software. Ao longo dos experimentos foi possível observar que esse modelo consegue gerar conjuntos de testes abrangentes, aplicando critérios clássicos como classes de equivalência, análise de valor limite, cobertura de linha, cobertura de ramificação e cobertura de condições de forma consistente. A aplicação correta desses critérios foi confirmada pelos altos índices de cobertura estrutural, que mantiveram próximos de de 98,1% para instrução e 98% para ramificações, bem como pelos níveis satisfatórios de cobertura de mutação (média de 91,1%), que refletem a capacidade dos testes gerados em identificar defeitos potenciais.

A qualidade dos casos de testes produzidos é, de maneira geral, elevada, especialmente quando os critérios envolvem estruturas lógicas bem definidas, com casos da

cobertura de condições. Contudo, desafios foram observados em cenários que dependem de valores críticos, como na análise de valor limite, e em programas com regras mais complexas, como CountWords e SubstringsBetween, onde houveram ocorrências de casos inválidos e falhas. Esses problemas, embora pontuais, destacam uma limitação atual das LLMs, a dificuldade de interpretar completamente a especificação e gerar entradas corretamente alinhadas ao comportamento do software.

No que se refere a automação, os experimentos demonstram que as LLMs são capazes de gerar scripts prontos para execução em contextos menos complexos, como nos programas PlanningPoker e TriangleType, onde a automação foi integral e não exigiu ajustes. Em contrapartida, em programas mais elaborados, houve necessidade de intervenção manual para remover ou comentar casos inválidos e falhos, o que reduziu parcialmente os ganhos esperados com a automação proporcionando uma base sólida para execução, reduzindo significativamente o esforço humano em relação à criação manual de testes.

Com base na análise dos casos inválidos por programa revela o impacto direto na automação e na confiabilidade dos testes. O programa CountWords apresentou um cenário crítico, com 23,5% de casos inválidos, exigindo que 20 testes fossem comentados, o que comprometeu a automação e reduziu a efetividade do conjunto de testes. O SubstringsBetween teve um impacto moderado, com 6,60% de casos inválidos que também exigiram ajustes manuais. Em contraste, PlanningPoker e TriangleType obtiveram o cenário ideal, com zero casos inválidos, permitindo execução completa e automação integral. Esses dados indicam que a confiabilidade é fortemente afetada por inconsistências nos casos gerados, tornando a revisão humana indispensável para garantir a eficiência do processo e evitar uma falsa percepção de qualidade.

A análise foca no impacto das falhas por programa, mostra que, mesmo em cenários de alta cobertura estrutural, falhas podem ocorrer e comprometer a confiabilidade do processo. O SubstringsBetween apresentou a maior taxa de falhas (3,77%), com 4 casos que não produziram os resultados esperados, o que sugere a presença de comportamentos divergentes no código ou inconsistências nas instruções da LLM. O CountWords também evidenciou um impacto relevante, com 3,53% de falhas , reforçando a ideia de que códigos mais complexos tendem a gerar mais problemas. Por outro lado, o PlanningPoker obteve uma execução ideal sem falhas, e o TriangleType, apesar de uma falha, manteve um impacto baixo devido ao grande volume de casos válidos. A combinação desses fatores indica que, embora as LLMs possam gerar alta cobertura, a confiabilidade ainda é limitada por falhas inesperadas, o que reforça a necessidade de supervisão humana para um processo seguro.

De forma abrangente, os resultados confirmam que as LLMs podem atuar como ferramentas eficazes no processo de teste, tanto no planejamento e geração de casos quanto na automação parcial ou total da execução. Embora ainda exista a necessidade de supervisão e ajustes, principalmente para garantir conformidade com a especificação, a utilização de LLMs apresenta um potencial significativo para aumentar a produtividade, reduzir custos e ampliar a cobertura dos testes, aproximando-se cada vez mais de uma solução prática para cenários reais de desenvolvimento de software.

6.4 Limitações do Estudo

Apesar dos resultados promissores, este estudo apresenta algumas limitações que devem ser consideradas. A primeira refere-se ao escopo restrito, pois os experimentos foram conduzidos com apenas quatro programas provenientes de livros didáticos. Embora tenham sido escolhidos por sua diversidade estrutural, eles não representam plenamente os desafios de sistemas reais, que possuem maior complexidade, integração e variabilidade de cenários.

Outra limitação diz respeito ao uso de apenas um LLM, o Gemini Pro, o que impossibilita comparações com outros modelos que poderiam apresentar diversos níveis de precisão, estabilidade e capacidade de compreensão. Essa restrição impede conclusões mais amplas sobre qual abordagem seria mais adequada em diferentes contextos.

Por fim, destaca-se a aleatoriedade inerente aos LLMs, que faz com que, mesmo utilizando os mesmos prompts, resultados distintos possam ser obtidos em execuções repetidas de um mesmo prompt. Essa característica impacta a reprodutibilidade e pode representar um obstáculo para a adoção industrial, onde a previsibilidade é fundamental.

Essas limitações não comprometem os resultados, mas indicam oportunidades para estudos futuros, como ampliar a base de programas, testar diferentes modelos e investigar estratégias que reduzem a variabilidade nos resultados.

7 CONSIDERAÇÕES FINAIS

Este trabalho teve como objetivo principal investigar a aplicação de LLMs no contexto de teste de software, avaliando sua capacidade de gerar casos de testes e scripts automatizados com base em critérios da literatura. A pesquisa foi conduzida por meio de um estudo experimental utilizando o Gemini Pro 2.5 como modelo base e aplicando técnicas de teste funcionais (particionamento de classe e análise de valor limite) e estruturais (cobertura de linhas, cobertura de ramificações e cobertura de condições), além da geração de scripts para testes automatizados.

Os resultados obtidos evidenciam que os LLMs apresentam potencial significativo para apoiar atividades de garantia de qualidade, especialmente em fases críticas do processo de teste. Em termos quantitativos, os casos de testes gerados atingiram altos índices de cobertura estrutural, com médias próximas a 98,1% para cobertura de instruções e 98% de ramificações, valores que demonstram a efetividade das técnicas aplicadas. Adicionalmente, a análise baseada em teste de mutação indicou uma média de 91,1% de mutação mortos, o que reforça a capacidade dos conjuntos de testes gerados em detectar falhas no código.

Entretanto, a pesquisa também revelou limitações. Foram observados casos inválidos e falhas em entradas críticas, indicando que os LLMs ainda enfrentam desafios na compreensão de requisitos complexos e na interpretação precisa dos cenários. A automação total não se mostrou viável para códigos com maior nível de complexidade, exigindo intervenções manual para ajustes e validação dos artefatos gerados.

Em síntese, a aplicação de LLMs em teste de software é uma abordagem promissora, capaz de ampliar a produtividade e a cobertura dos testes, mas que ainda demanda estratégias complementares para garantir maior precisão e confiabilidade. Com a evolução contínua dessas ferramentas e o refinamento as práticas de engenharia de prompt.

7.1 Trabalhos Futuros

Com base no estudo realizado, os trabalhos futuros buscarão aprofundar a investigação sobre a aplicação de LLMs em teste de software, transformando as limitações identificadas em propostas concretas de pesquisa.

Uma das propostas é ampliar o escopo dos experimentos, utilizando sistemas mais complexos e com maior volume de código, extraídos de cenários reais da indústria de software. A ideia é investigar como as LLMs se comportam diante de aplicações complexas, compostas por múltiplos módulos, regras de negócio elaboradas e fluxos interdependentes.

Essa análise permitirá avaliar a escalabilidade da abordagem, a capacidade dos modelos e lidar com requisitos extensos e a viabilidade da automação em um contexto que exige maior precisão e controle.

Outra proposta é comparar a performance do Gemini Pro com outras LLMs, como o ChatGPT ou DeepSeek, para verificar diferentes níveis de precisão, estabilidade e capacidade de compreensão. Essa comparação é importante para extrair conclusões mais amplas sobre qual modelo seria mais adequado em diferentes contextos dos testes.

Por fim, será importante investigar estratégias para reduzir a variabilidade inerente aos LLMs. O objetivo é buscar métodos que aumentem a reprodutibilidade dos resultados, um aspecto fundamental para adoção industrial da abordagem. Com essas iniciativas, busco contribuir para o entendimento do papel dos Lms no processo de Teste de Software, fornecendo evidências práticas e recomendações que possam auxiliar sua aplicação em projetos reais de desenvolvimento.

8 REFERÊNCIAS

SOMMERVILLE, I. **Engenharia de software.** 9. ed. São Paulo: Pearson, 2011.

AMMANN, Paul; OFFUTT, Jeff. **Introduction to software testing**. 2. ed. Cambridge: Cambridge University Press, 2017.

DELAMARO, Márcio Eduardo; MALDONADO, José Carlos; JINO, Mario. **Introdução ao teste de software**. Rio de Janeiro: Campus, 2007.

ANICHE, Mauricio. **Effective software testing: a developer's guide**. Shelter Island: Manning Publications, 2022.

WHITE, J.; HAYS, S.; FU, Q.; SPENCER-SMITH, J.; SCHMIDT, D. C. **ChatGPT promptpatterns for improving code quality, refactoring, requirements elicitation, and software design**. arXiv preprint arXiv:2303.07839, 2023.

BELZNER, Lenz; GABOR, Thomas; WIRSING, Martin. **Large language model assisted software engineering: prospects, challenges, and a case study preprint**. Cham: Springer, 2023. (Lecture Notes in Computer Science, v. 14380).

LIU, Z. et al. **Make LLM a testing expert:** Bringing Human-like Interaction to Mobile GUI Testing via Functionality-aware Decisions. In: ICSE ’24 – 46th International Conference on Software Engineering, Lisbon, Portugal, 2024.

LIU, K. et al. **LLM-powered test case generation for detecting tricky bugs.** Conference’24, April 2024, Washington, DC, USA.

SOMMERVILLE, I. **Engenharia de Software**; tradução Luiz Claudio Queiroz; revisão técnica Fábio Levy Siqueira. 10.ed., São Paulo: Person Education do Brasil, 2018.

JACOCO. JaCoCo - **Java Code Coverage Library:** counters. Disponível em: https://www.jacoco.org/jacoco/trunk/doc/counters.html. Acesso em: 29 jul. 2025.

PITEST. Pitest - **Mutation testing for java**. Disponível em: https://pitest-org.translate.goog/?_x_tr_sl=en&_x_tr_tl=pt&_x_tr_hl=pt&_x_tr_pto=tc. Acesso em: 29 jul. 2025.